Introduction to Deep Learning

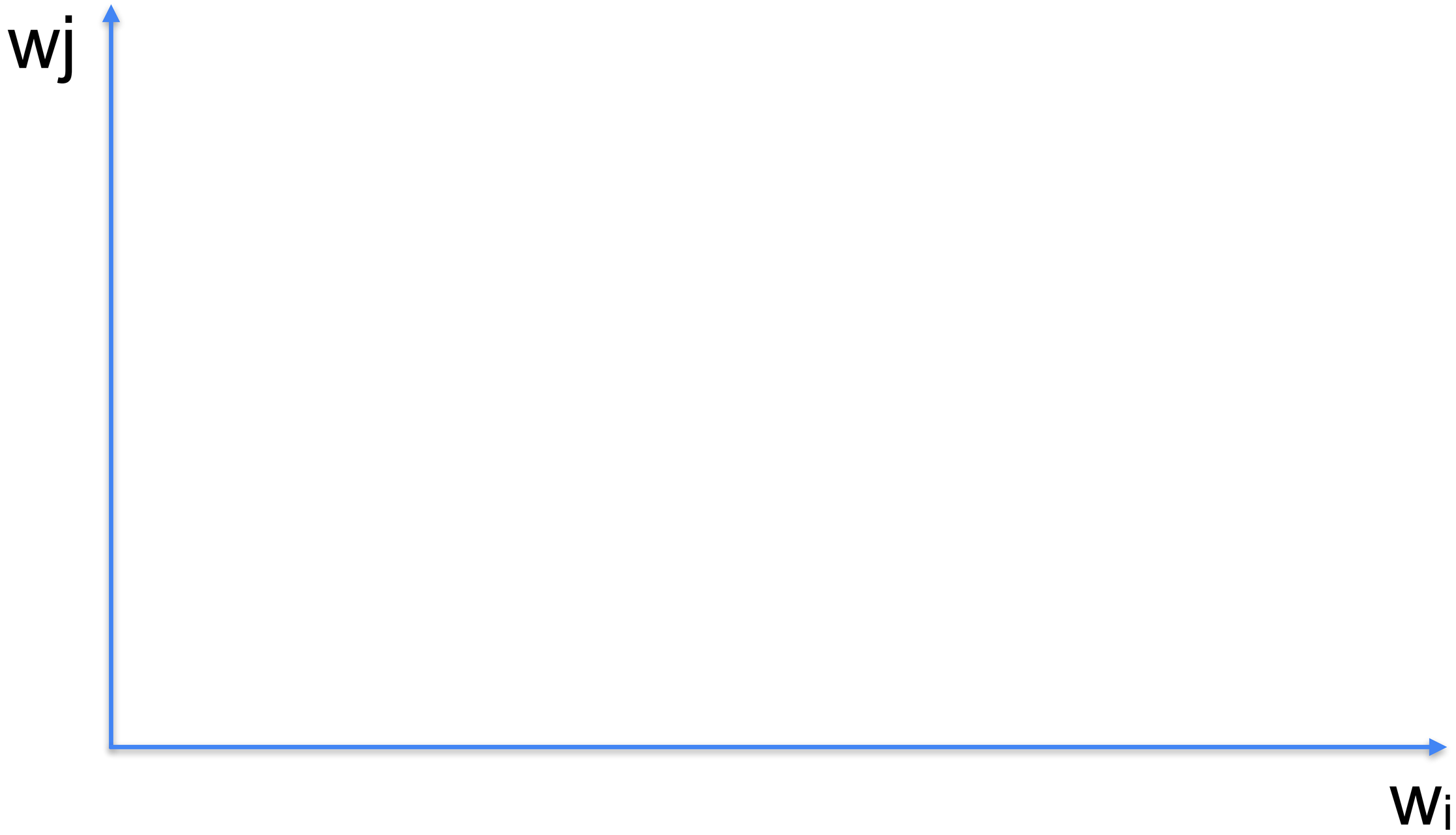
Caltech EE/CS/CNS 148a - Winter 2026
P. Perona

Lecture 4

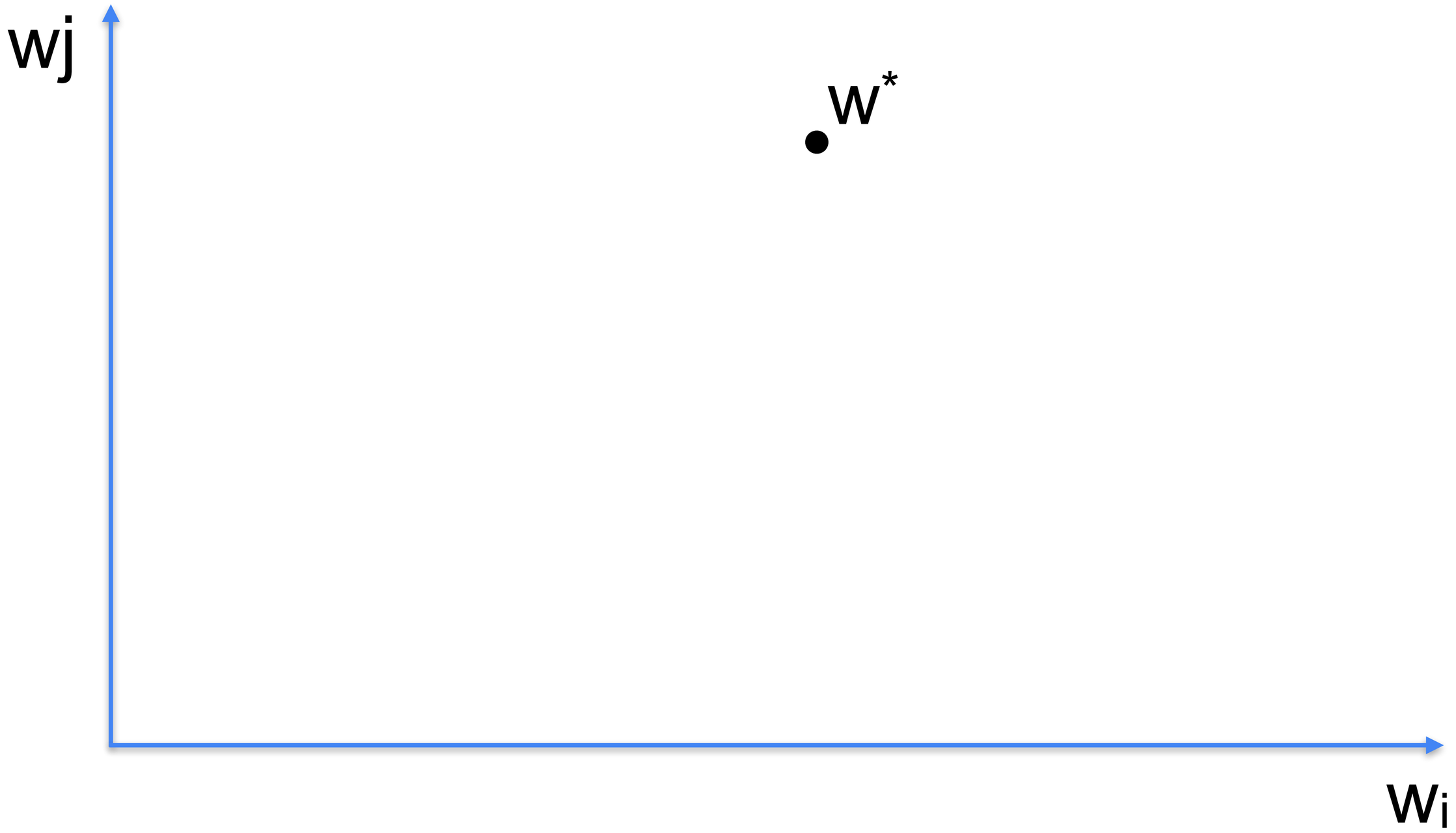
Step Size, Momentum, Adam, GPUs

Stochastic gradient descent

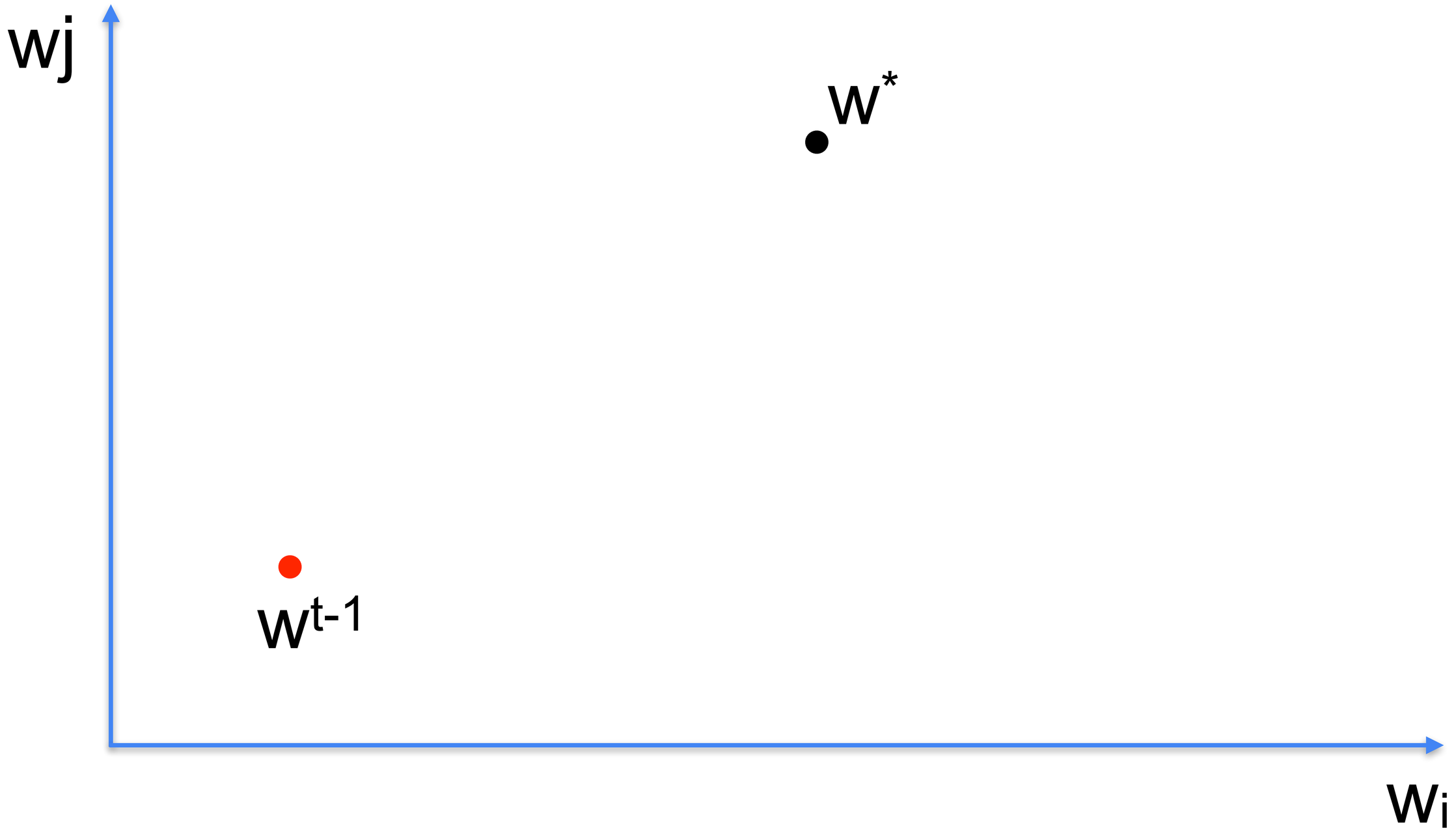
Gradient descent



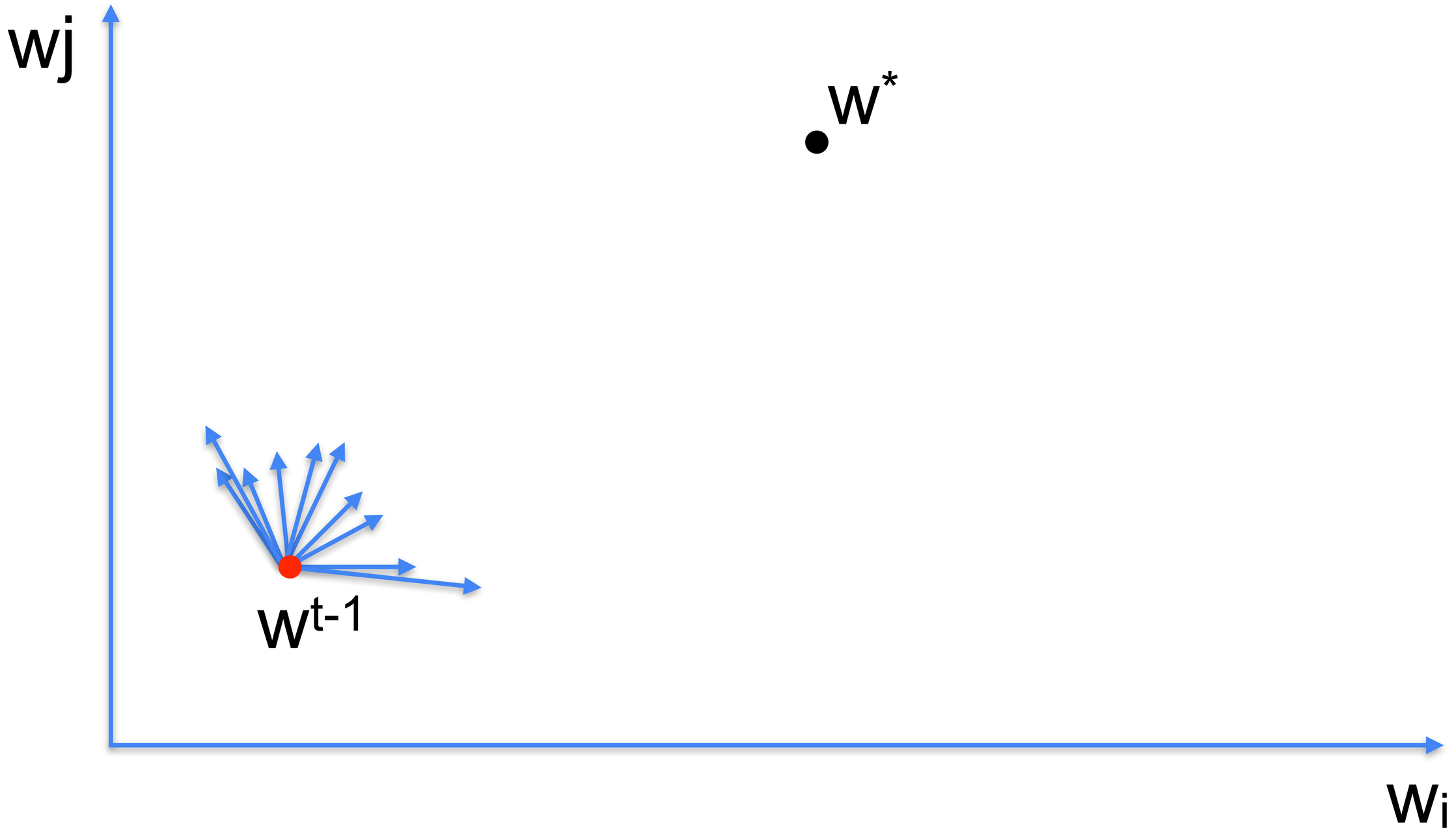
Gradient descent



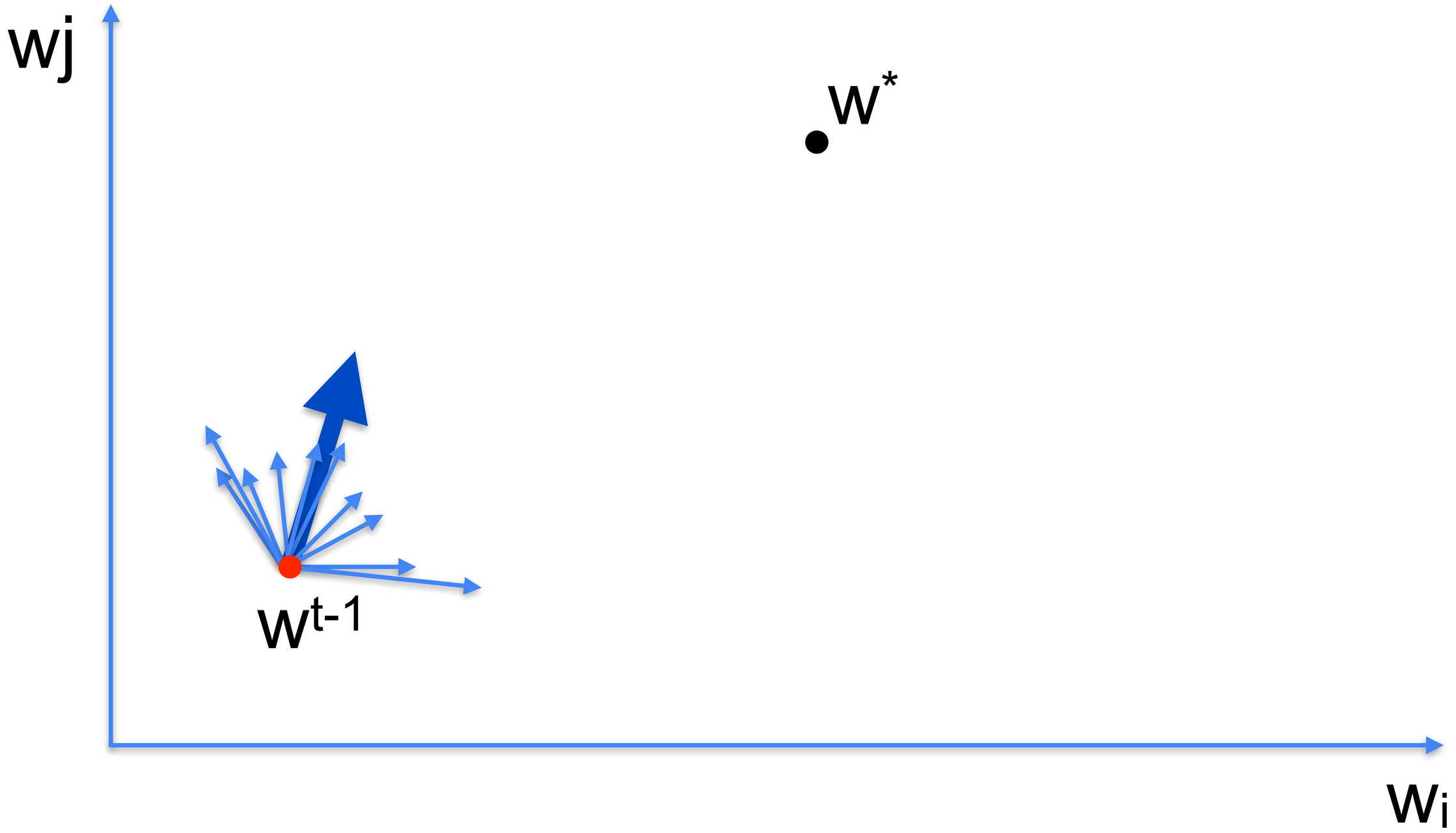
Gradient descent



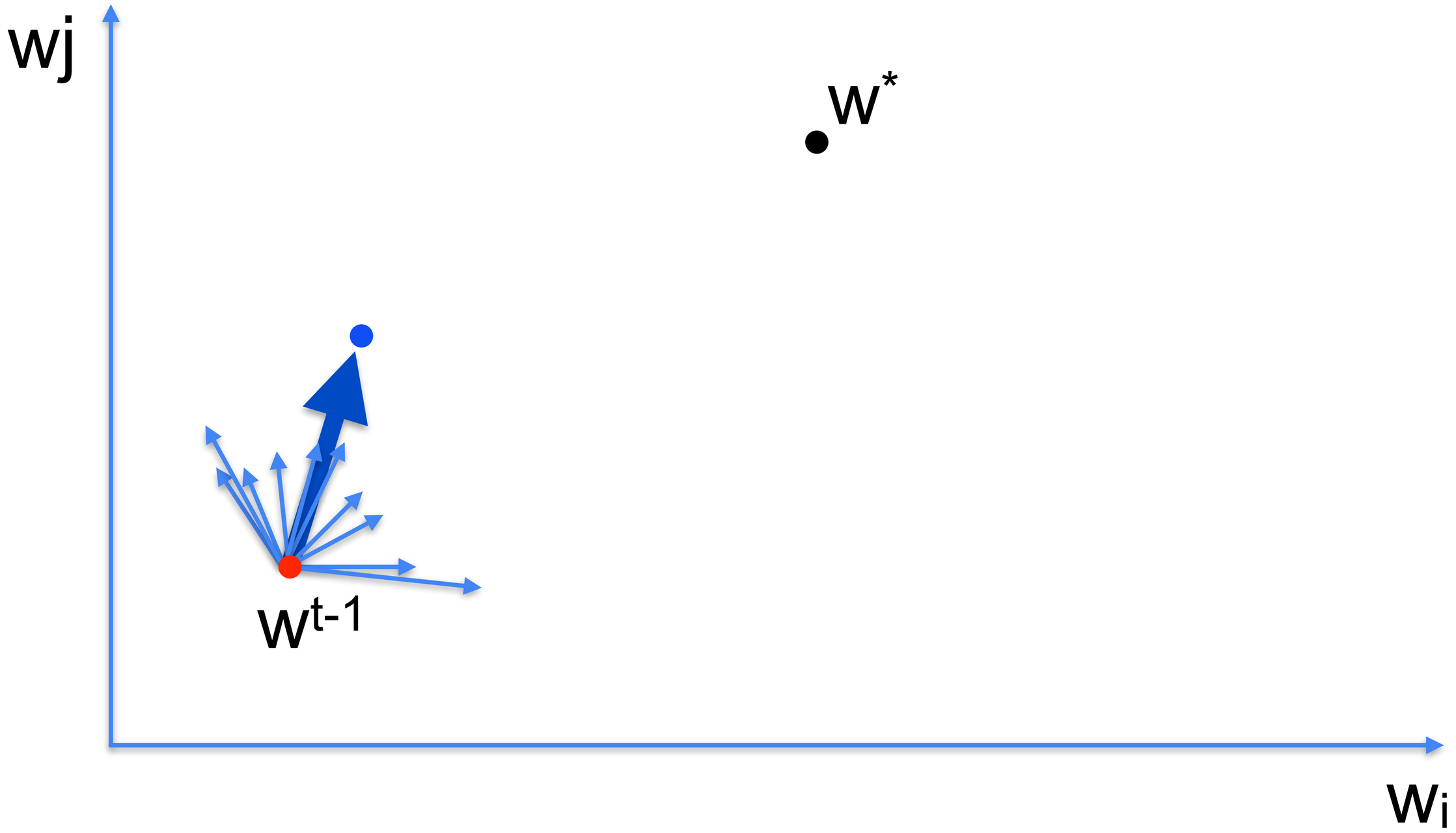
Gradient descent



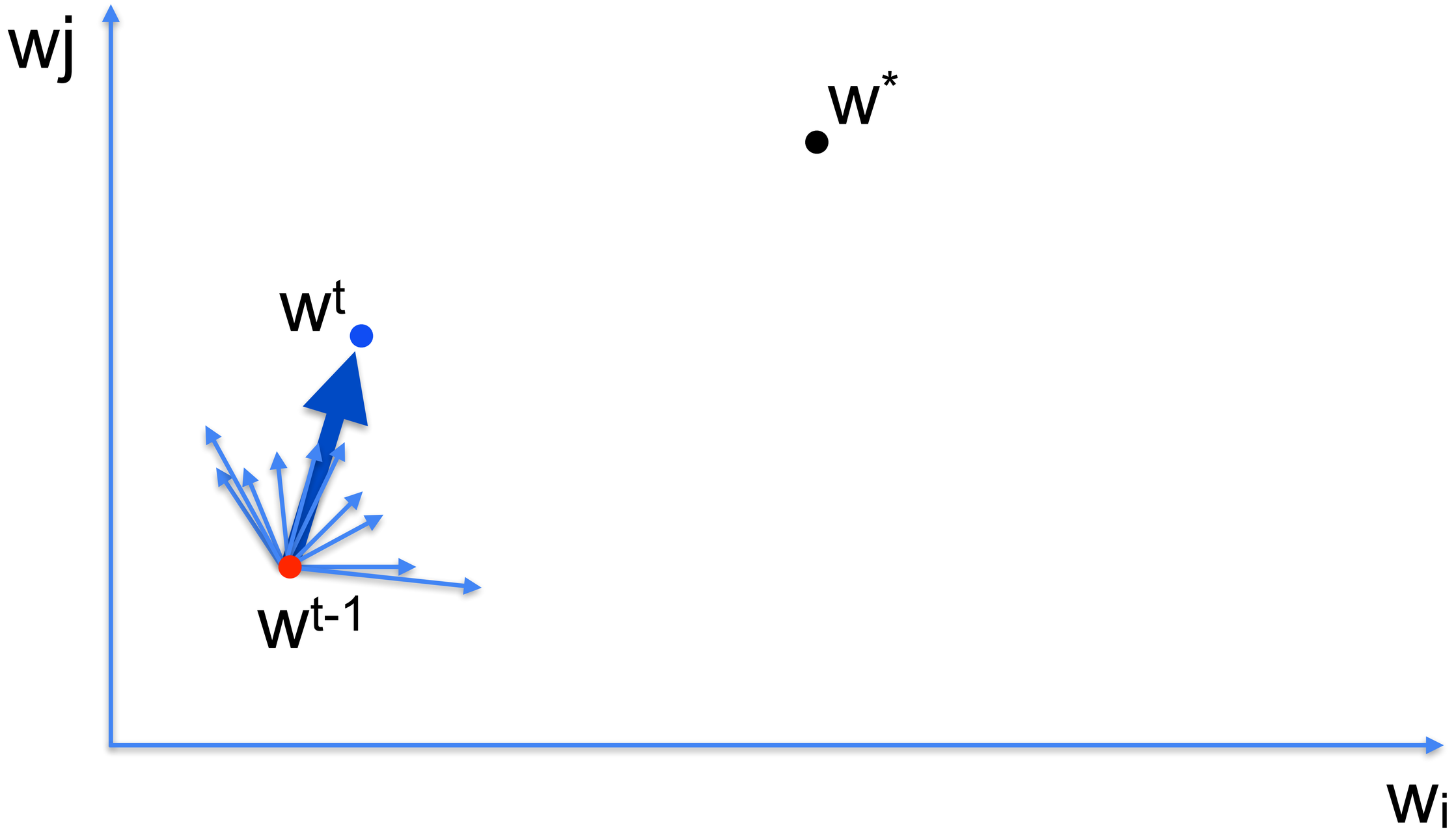
Gradient descent



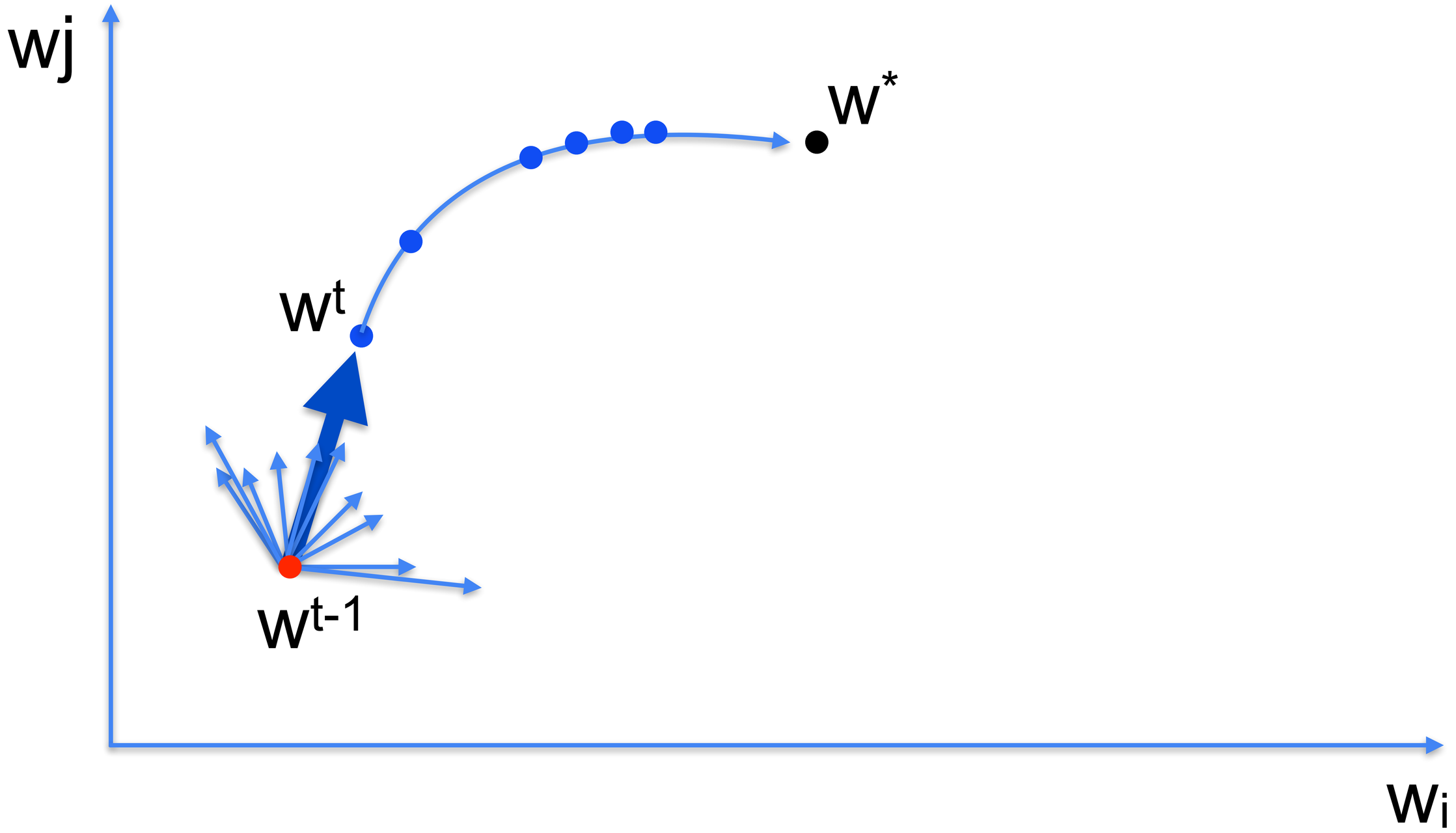
Gradient descent



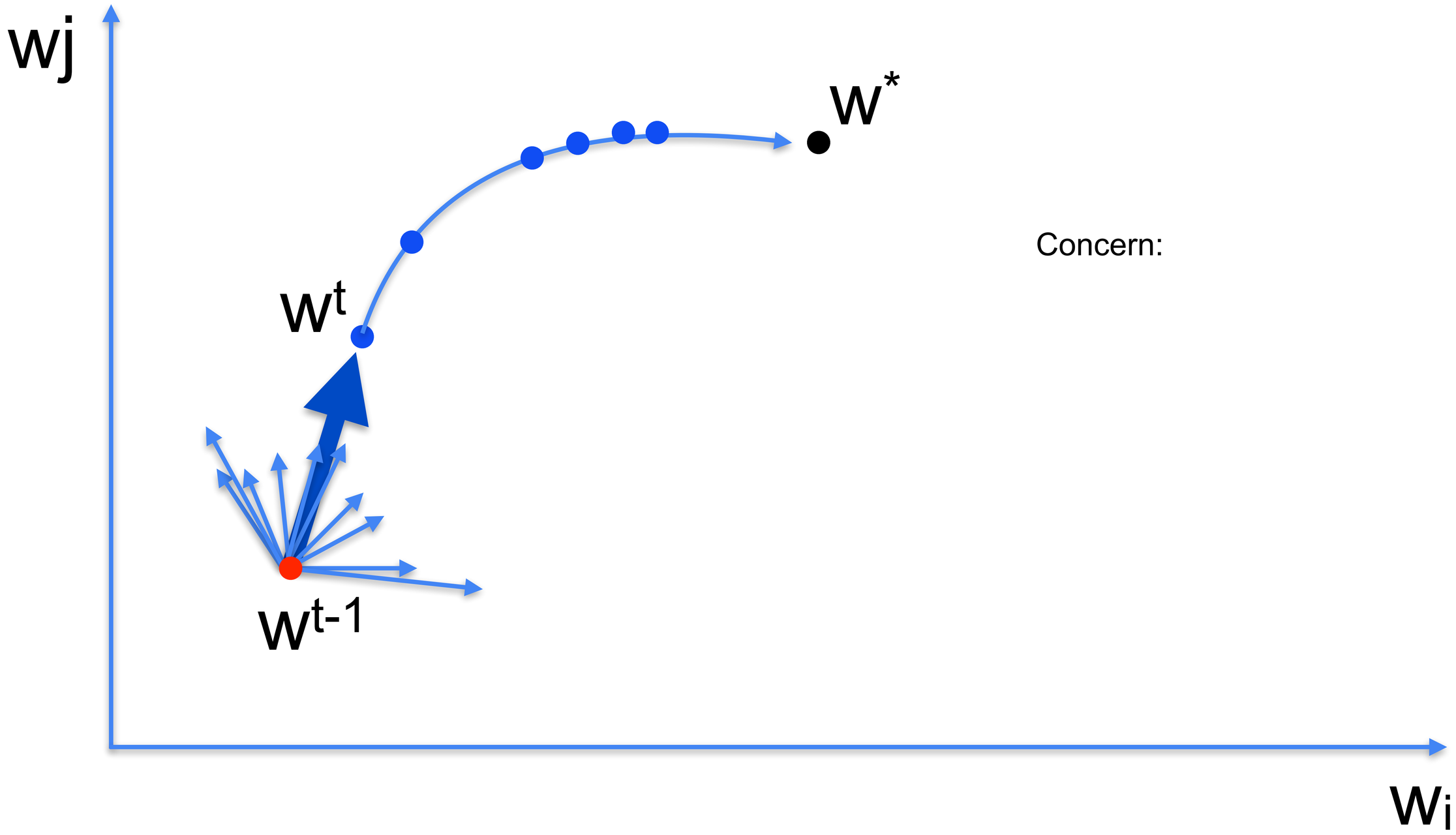
Gradient descent



Gradient descent

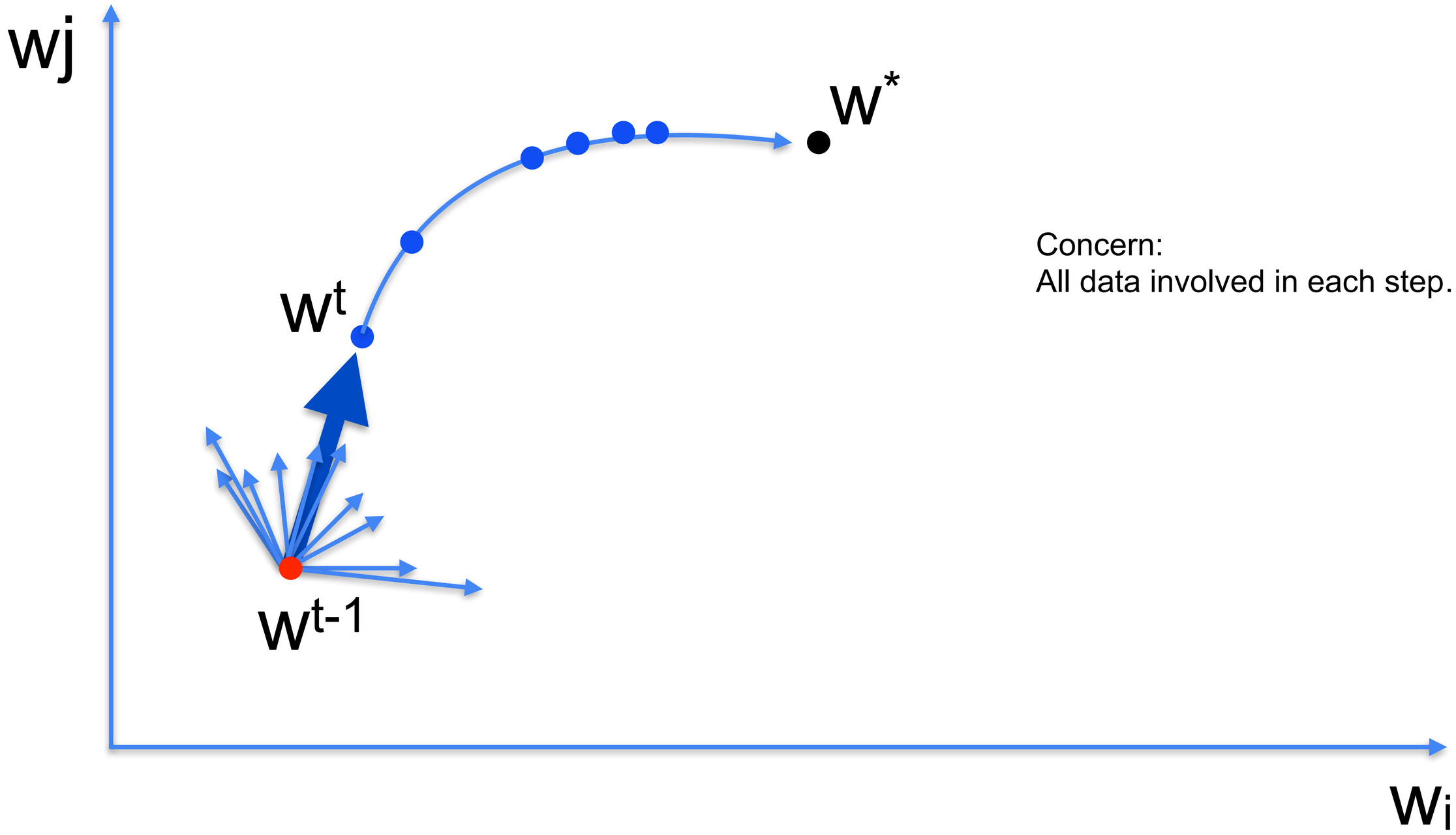


Gradient descent



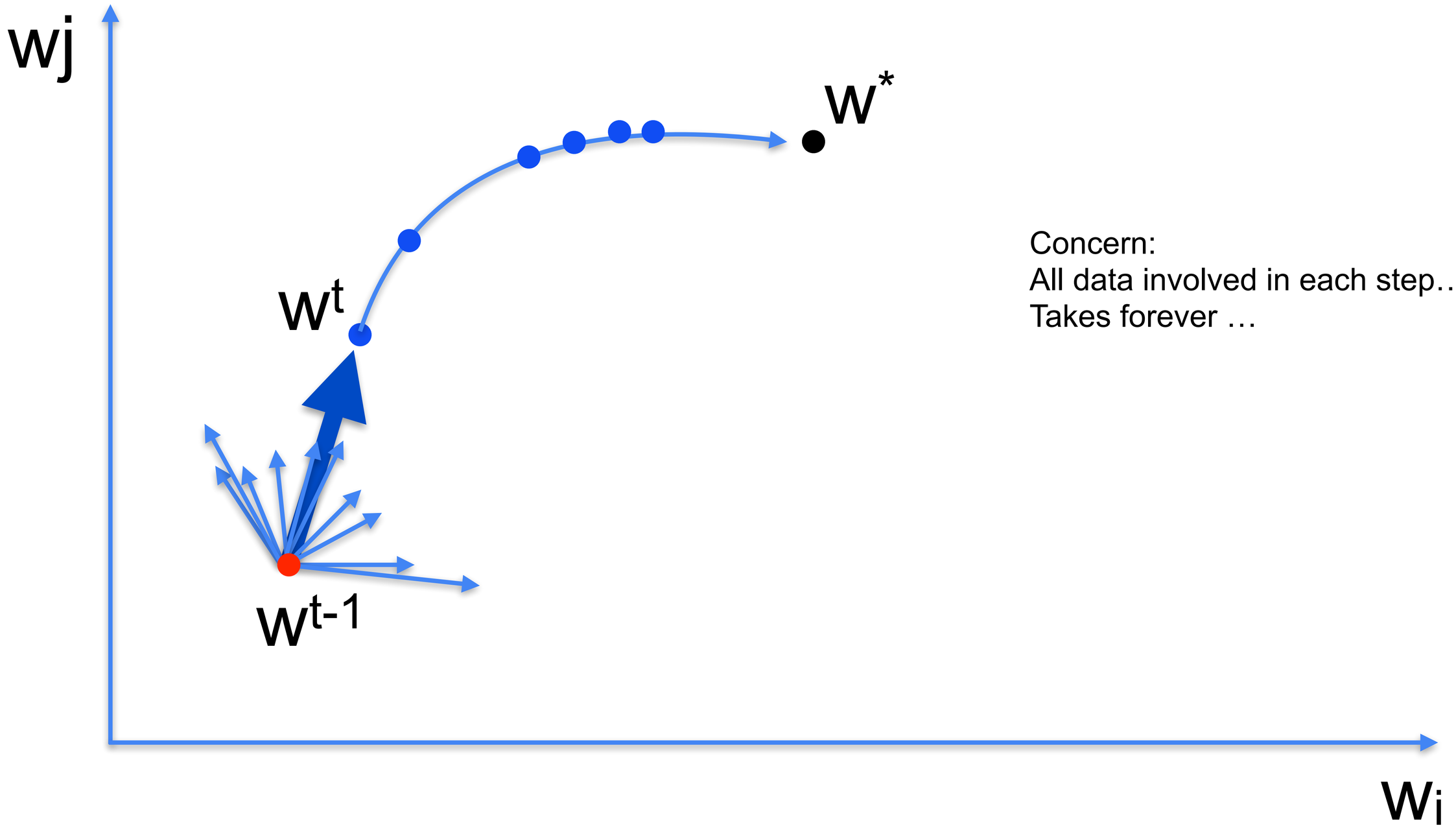
Concern:

Gradient descent



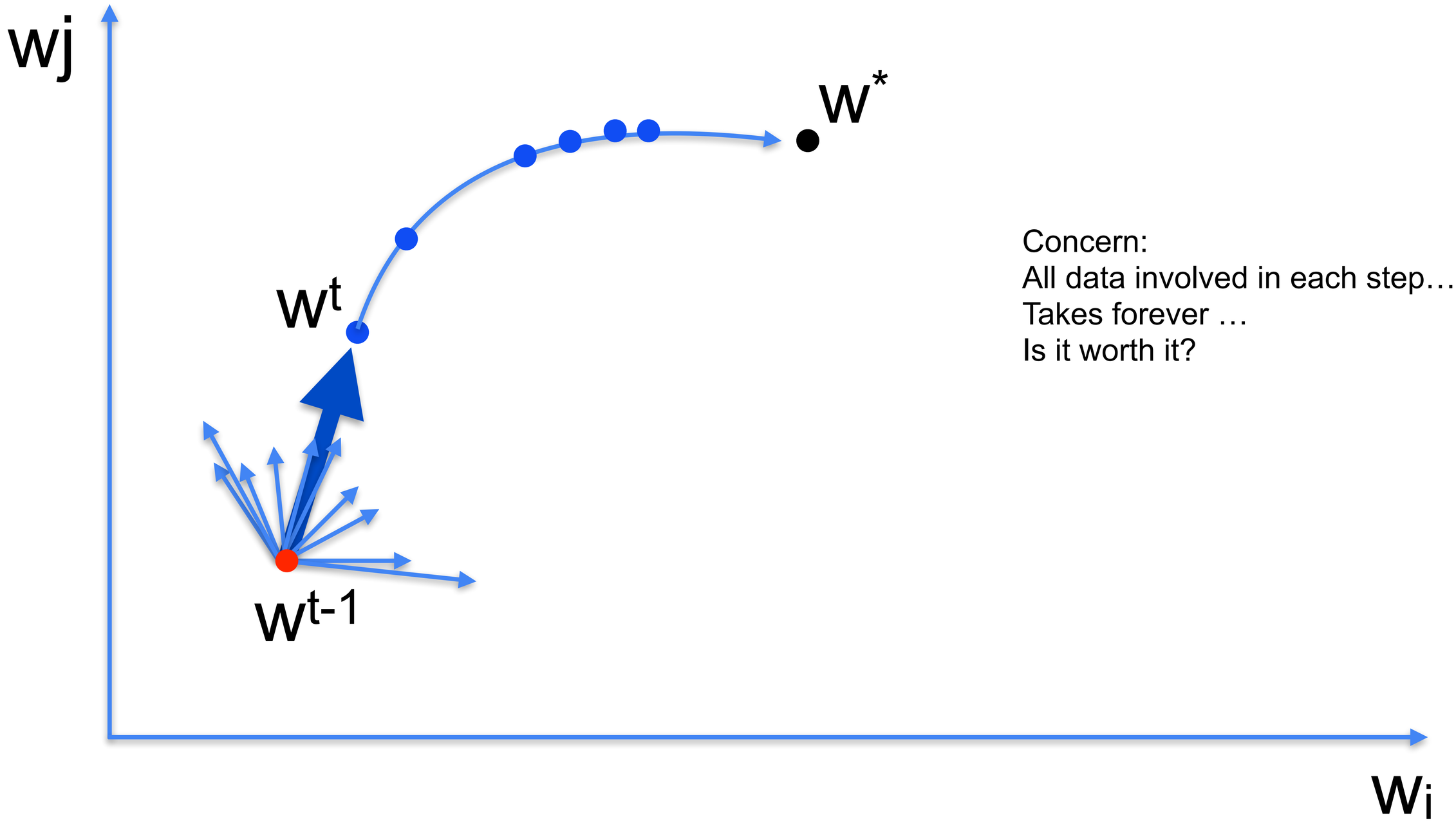
Concern:
All data involved in each step…

Gradient descent



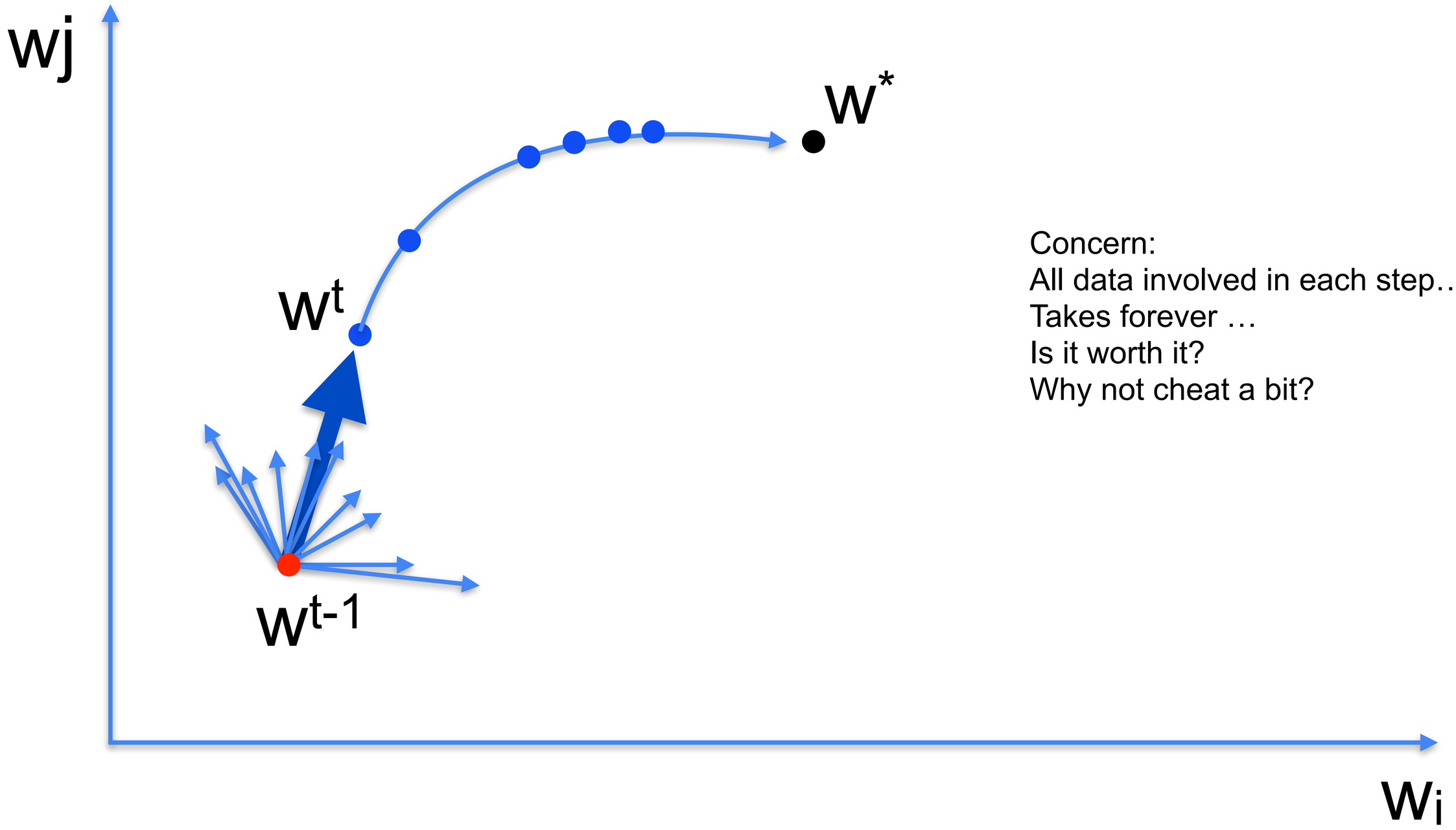
Concern:
All data involved in each step…
Takes forever …

Gradient descent



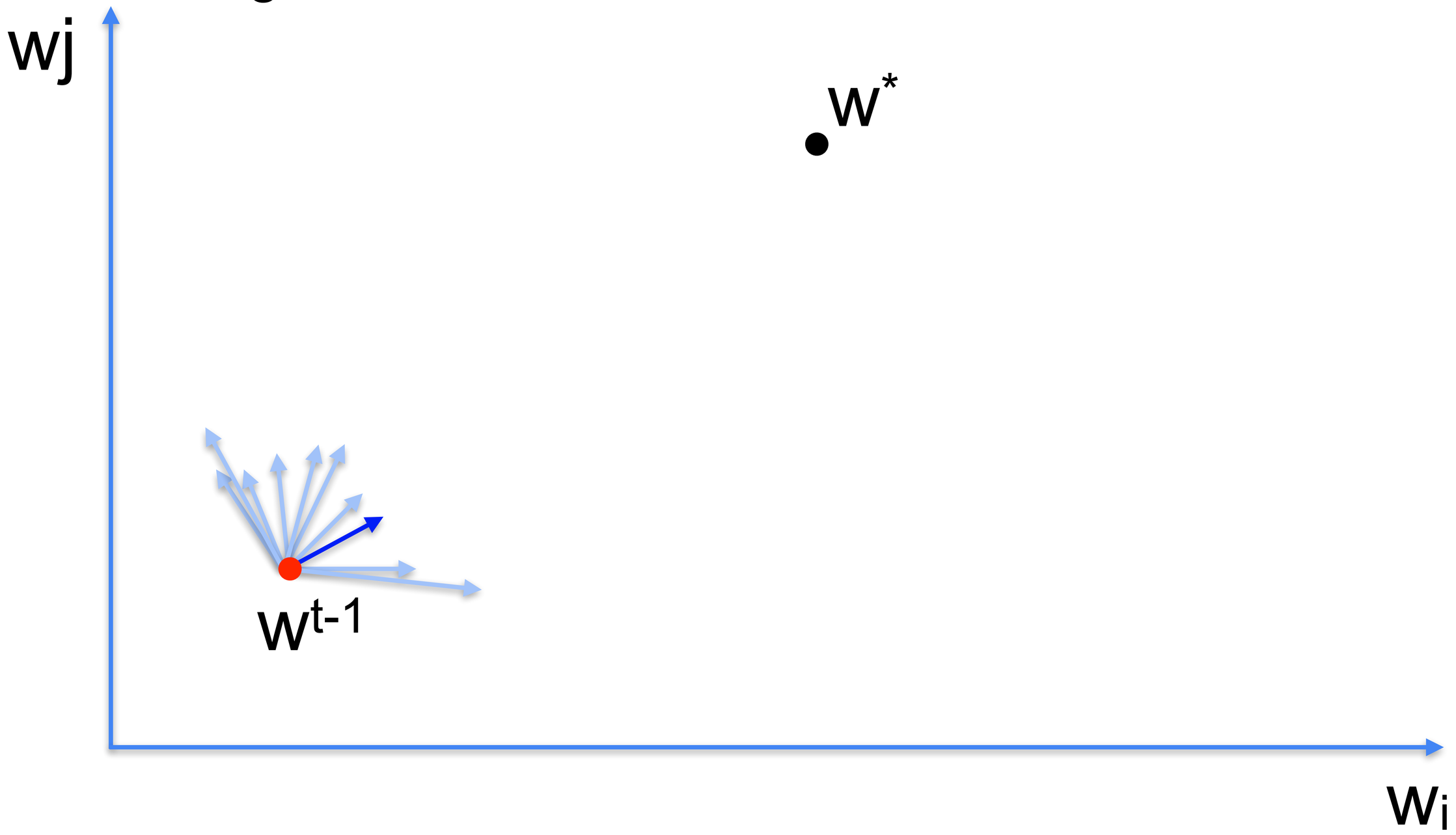
Concern:
All data involved in each step…
Takes forever …
Is it worth it?

Gradient descent

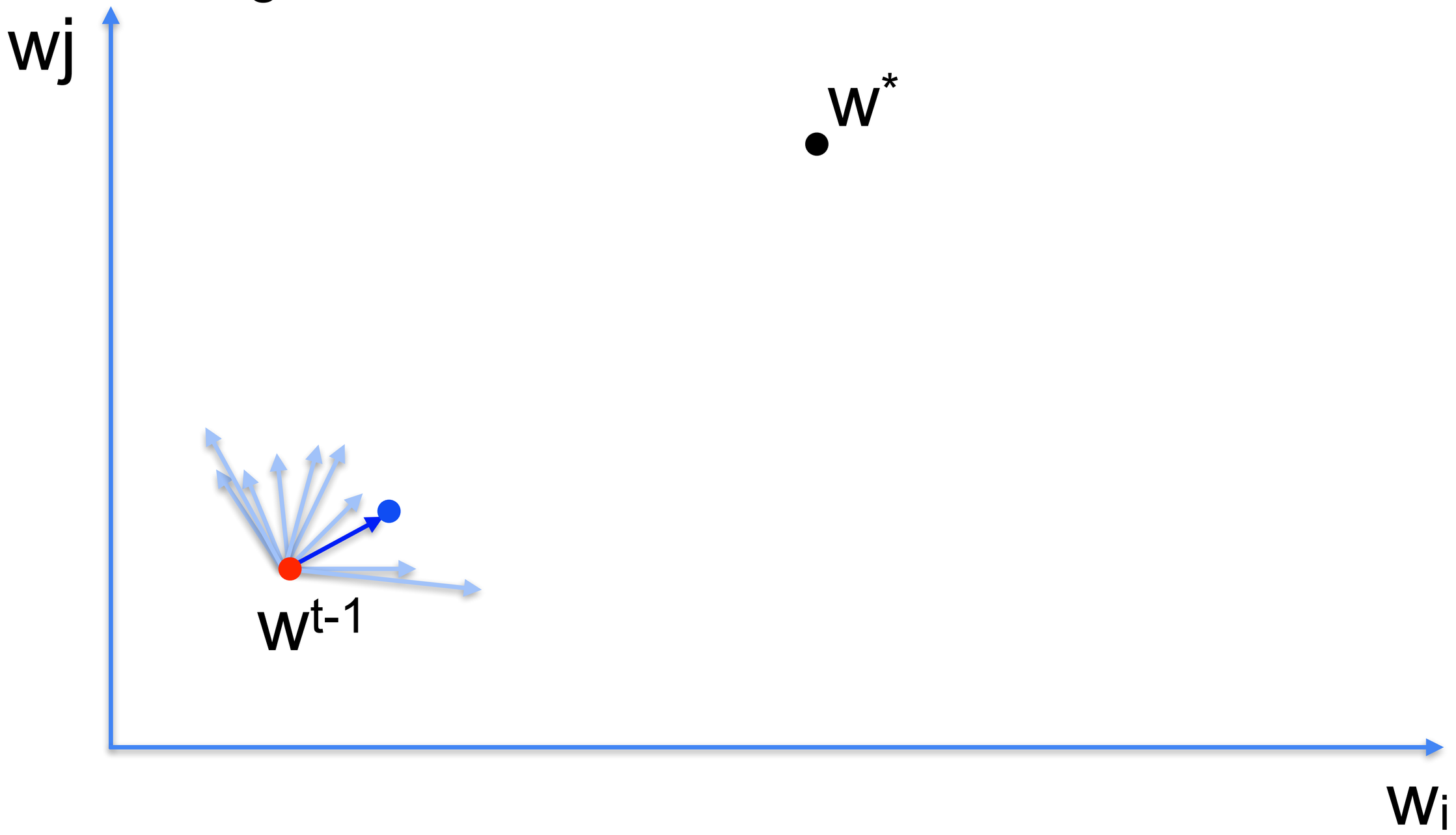


Concern:
All data involved in each step…
Takes forever …
Is it worth it?
Why not cheat a bit?

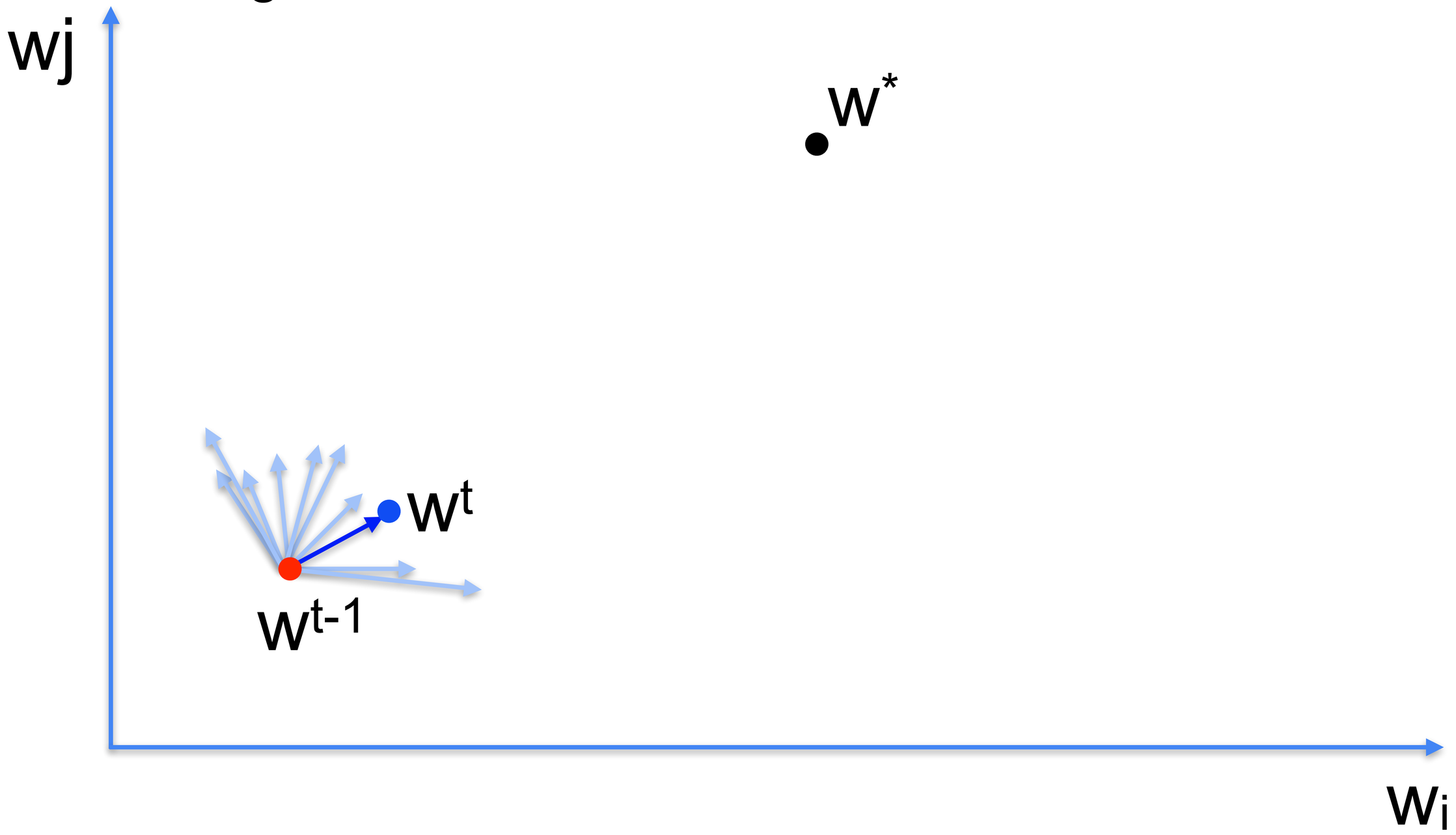
Stochastic gradient descent



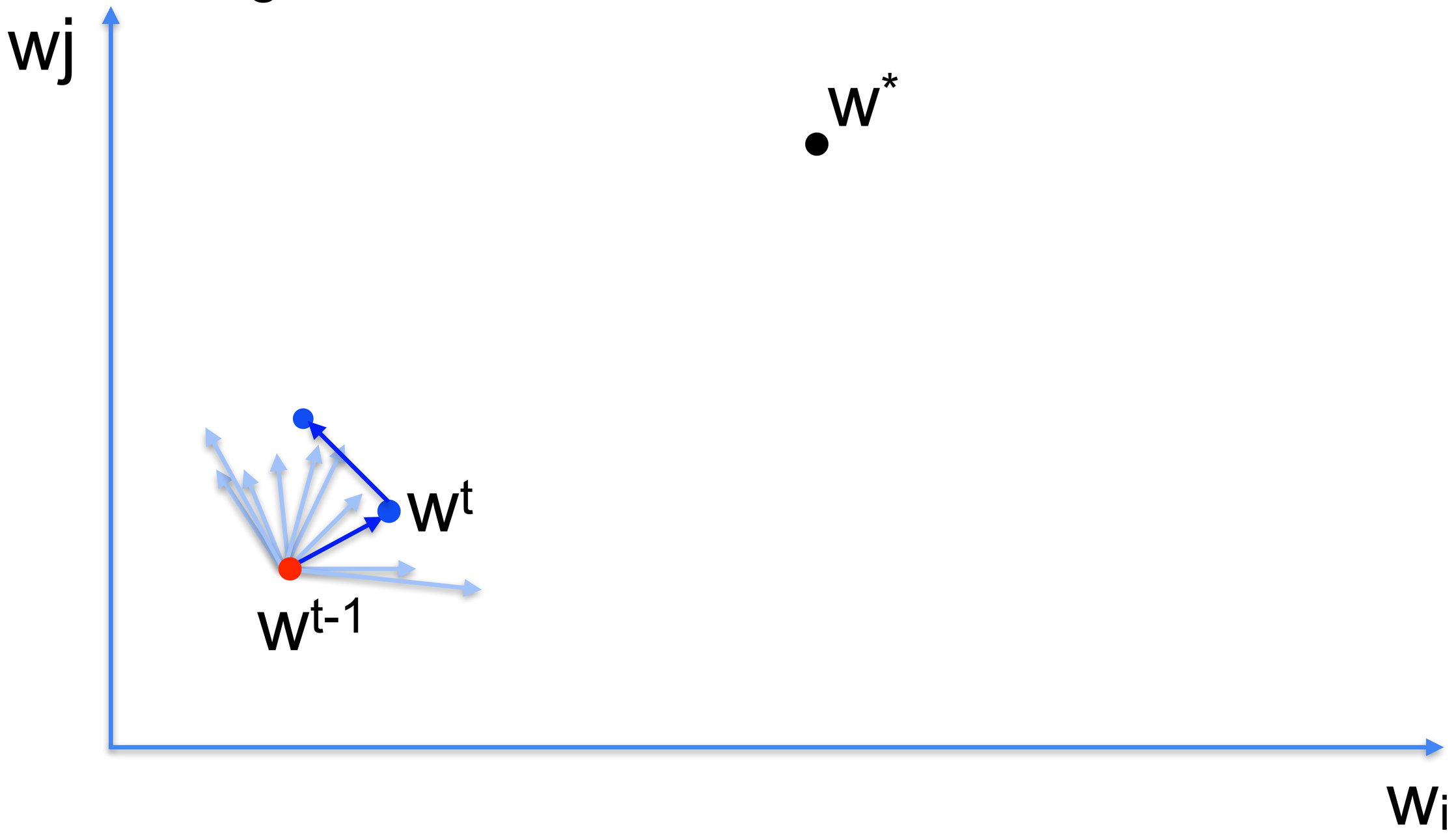
Stochastic gradient descent



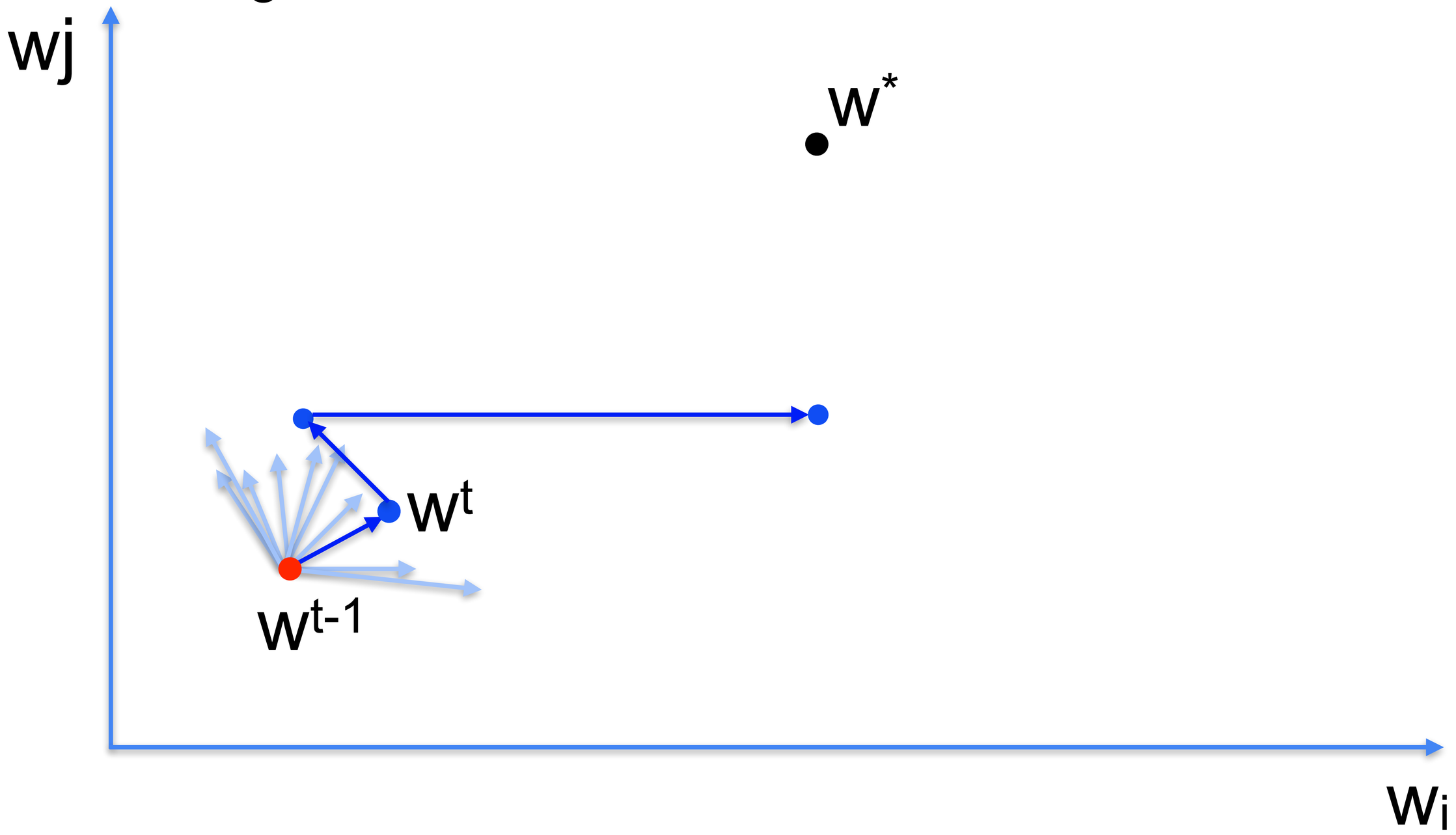
Stochastic gradient descent



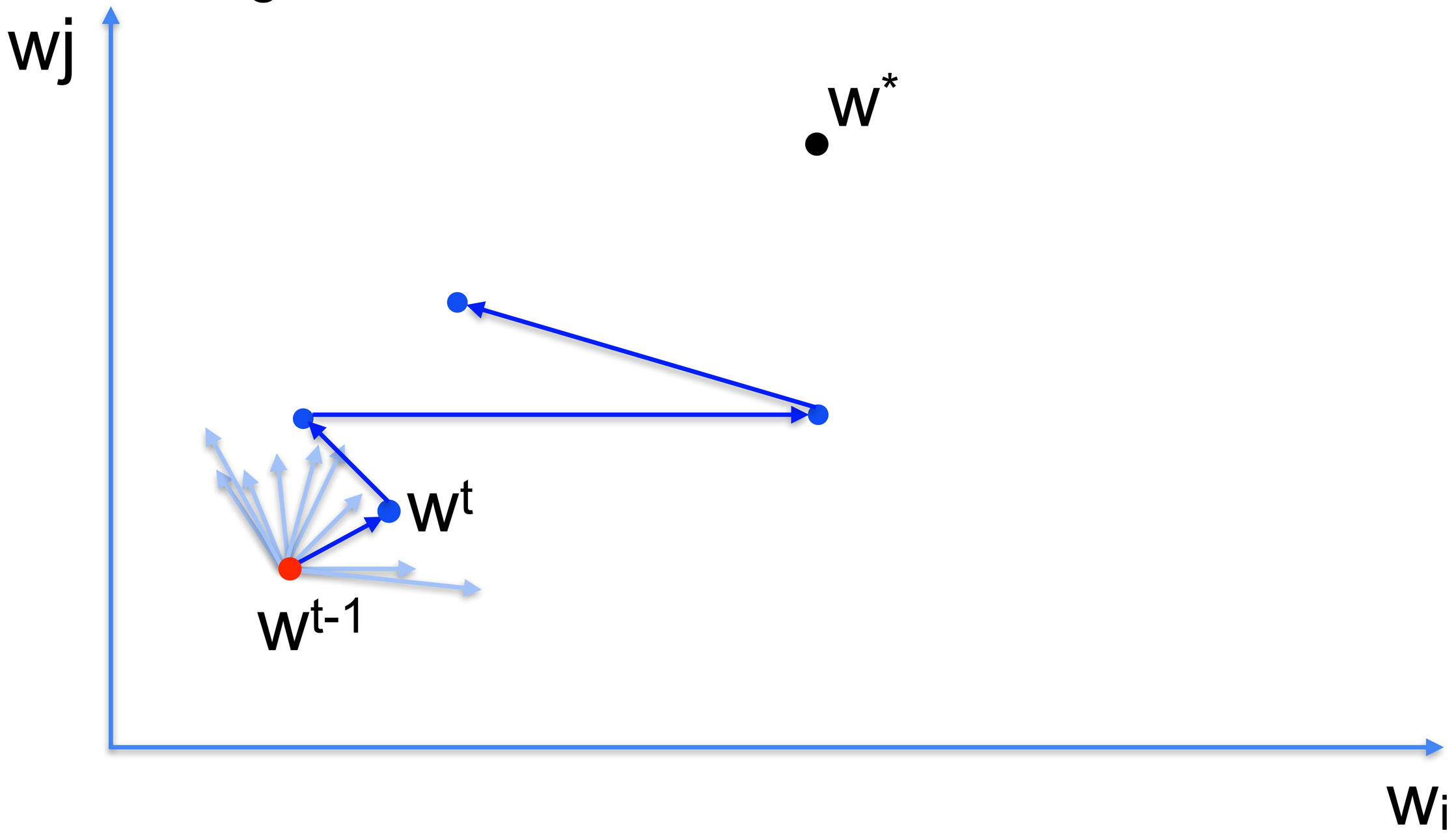
Stochastic gradient descent



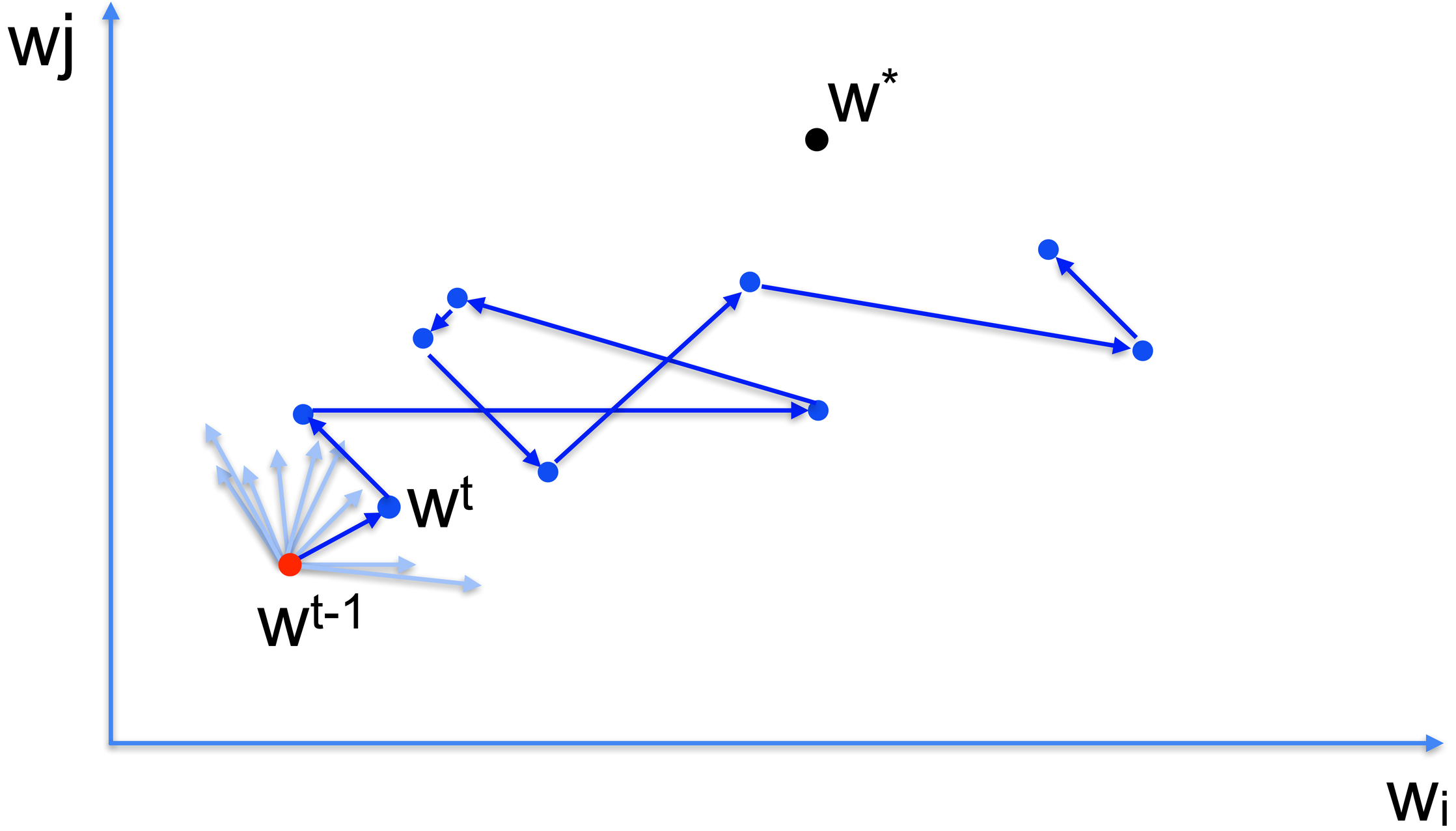
Stochastic gradient descent



Stochastic gradient descent



Stochastic gradient descent



Gradient descent

```
Input: dataset D = {(x(i), y(i))}, learning rate η, initial parameters w
repeat until convergence:
    compute gradient g = (1/N) * Σ_i ∇_w L(f(x(i), w), y(i))
    w ← w − η * g
return w
```

Gradient descent

```
Input: dataset D = {(x(i), y(i))}, learning rate η, initial parameters w
repeat until convergence:
    compute gradient g = (1/N) * Σ_i ∇_w L(f(x(i), w), y(i))
    w ← w − η * g
return w
```

Stochastic gradient descent (SGD)

```
Input: dataset D, learning rate η, initial parameters w
repeat until convergence:
    shuffle D
    for each (x(i), y(i)) in D:
        compute gradient g = ∇_w L(f(x(i), w) , y(i))
        w ← w − η * g
return w
```

Gradient descent

```
Input: dataset D = {(x(i), y(i))}, learning rate η, initial parameters w
repeat until convergence:
    compute gradient g = (1/N) * Σ_i ∇_w L(f(x(i), w), y(i))
    w ← w − η * g
return w
```

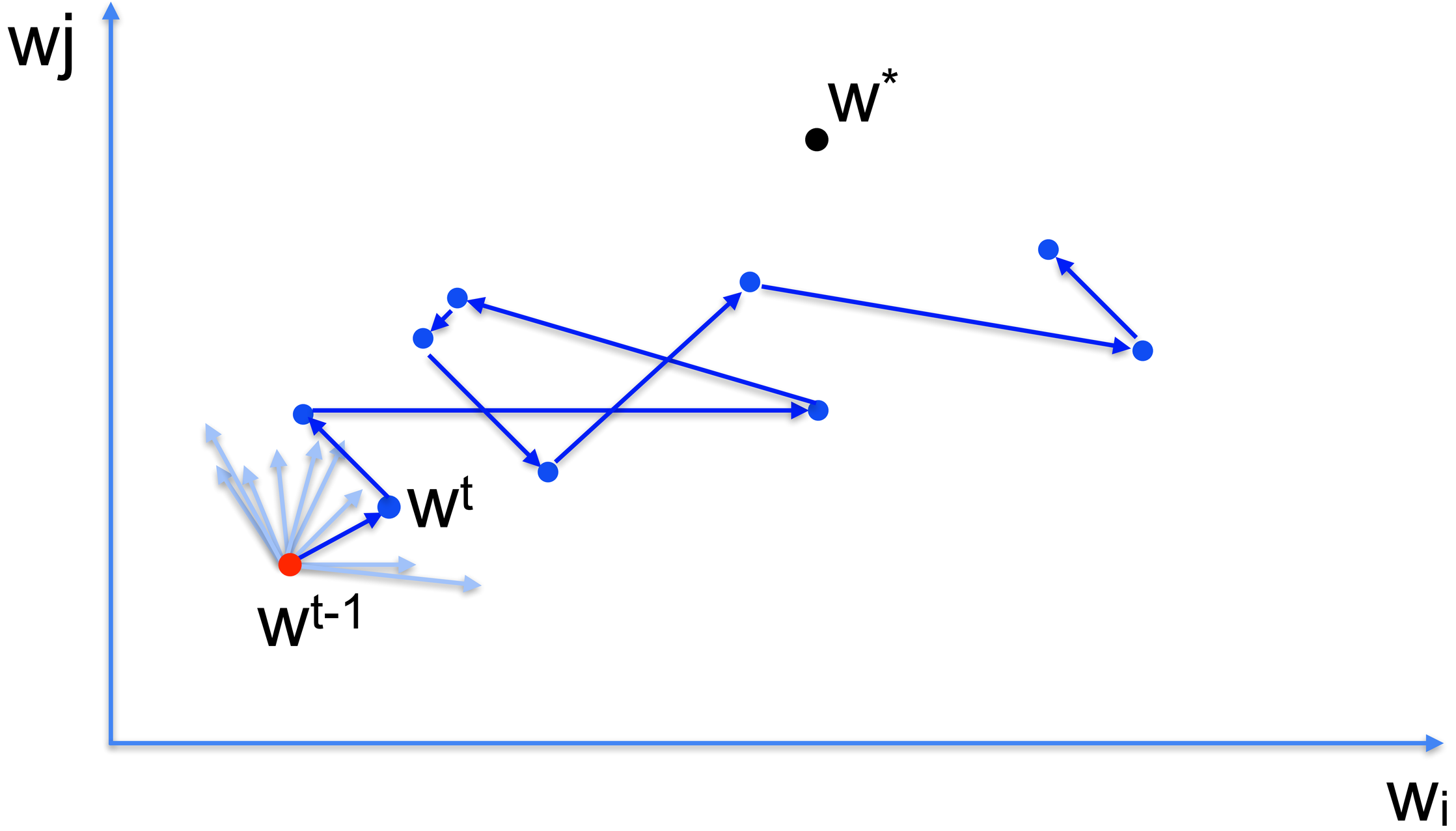
Stochastic gradient descent (SGD)

```
Input: dataset D, learning rate η, initial parameters w
repeat until convergence:
    shuffle D
    for each (x(i), y(i)) in D:
        compute gradient g = ∇_w L(f(x(i), w) , y(i))
        w ← w − η * g
return w
```

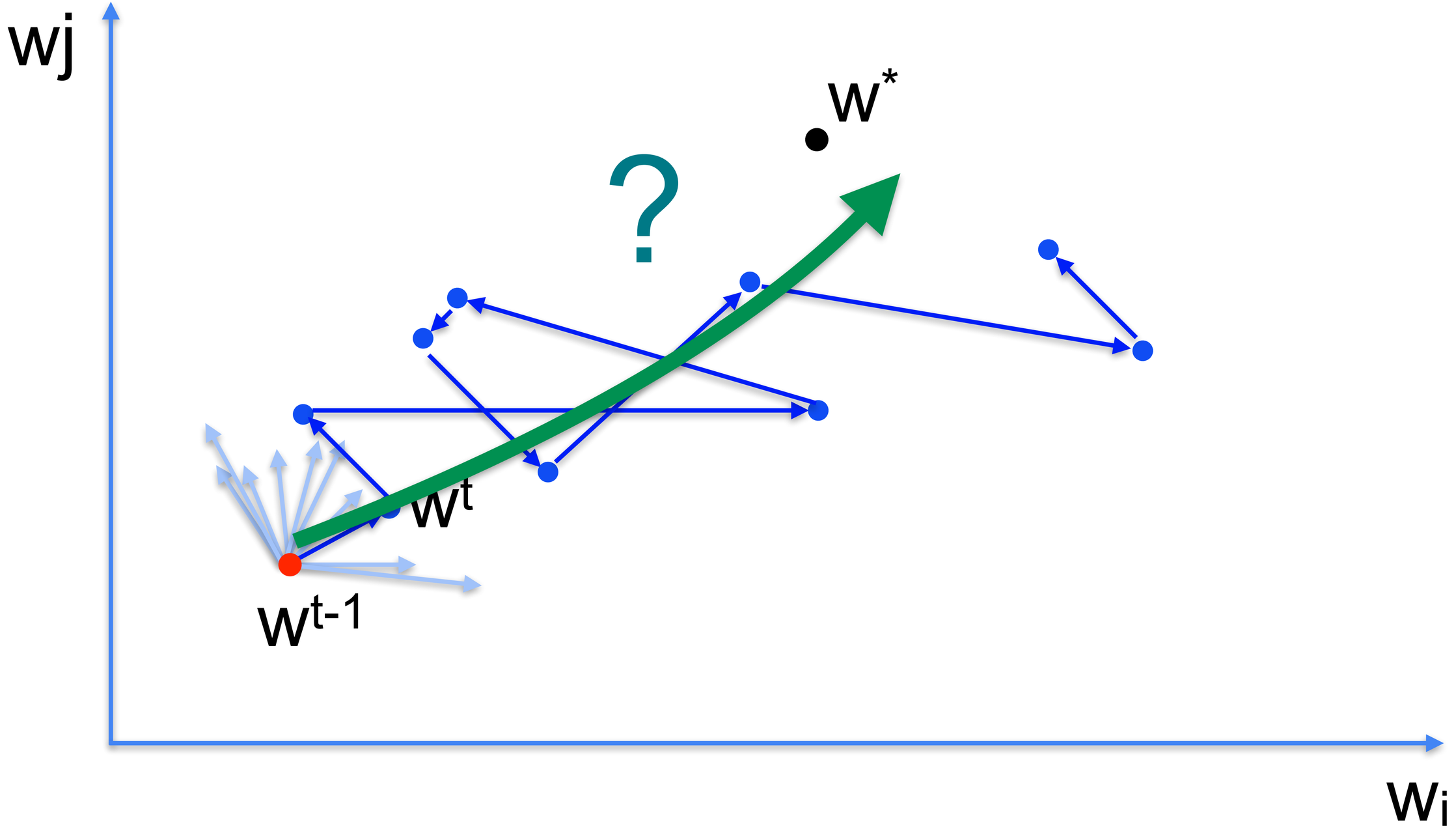
Mini-batch gradient descent

```
Input: dataset D, batch size B, learning rate η, initial parameters w
repeat until convergence:
    shuffle D
    partition D into batches of size B
    for each batch B_j:
        compute gradient g = (1/B) * Σ_(x,y)∈B_j ∇_w L(f(x, w), y)
        w ← w − η * g
return w
```

Idea for later: what about smoothing over time?



Idea for later: what about smoothing over time?



Learning rate

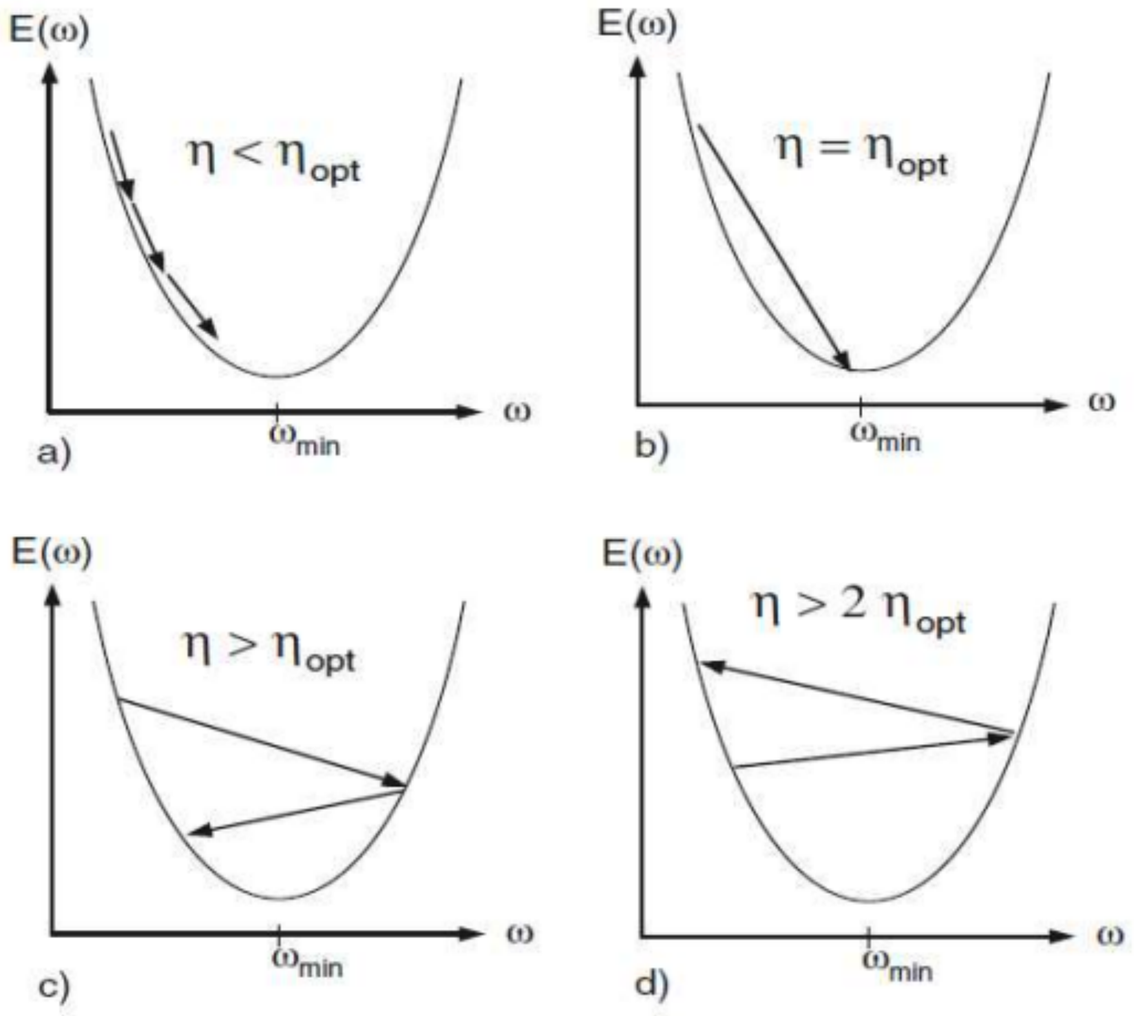
Learning rate

$$w_{t+1} = w_t - \eta \nabla \mathscr{L}(w_t)$$

From https://deeplearning.cs.cmu.edu/F24/index.html

Learning rate

$$w_{t+1} = w_t - \eta \nabla \mathscr{L}(w_t)$$

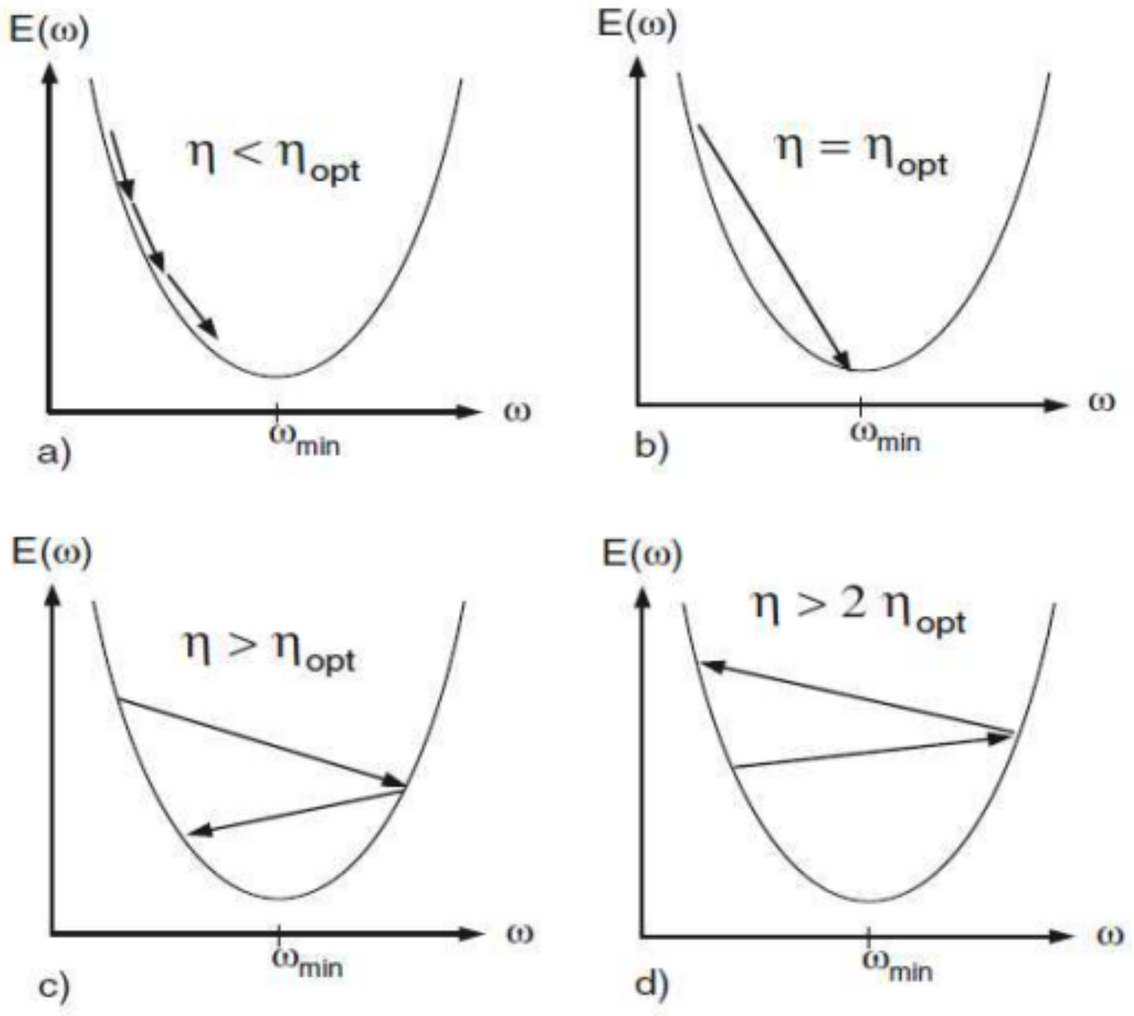


From https://deeplearning.cs.cmu.edu/F24/index.html

Learning rate

$$w_{t+1} = w_t - \eta \nabla \mathscr{L}(w_t)$$

η_{opt} ?



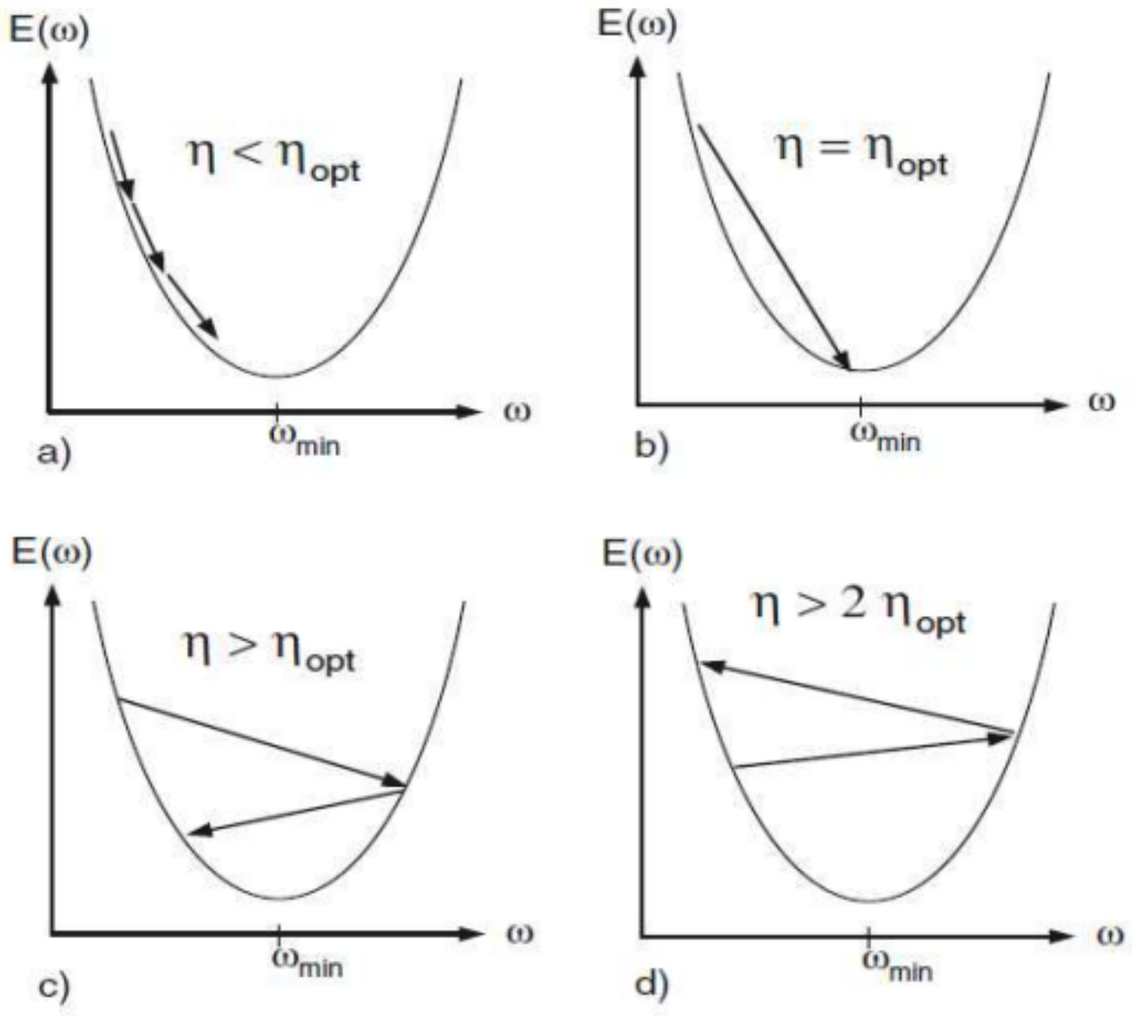
From https://deeplearning.cs.cmu.edu/F24/index.html

Learning rate

$$w_{t+1} = w_t - \eta \nabla \mathscr{L}(w_t)$$

η_{opt} ?

Hint: Taylor expansion around the current point



From https://deeplearning.cs.cmu.edu/F24/index.html

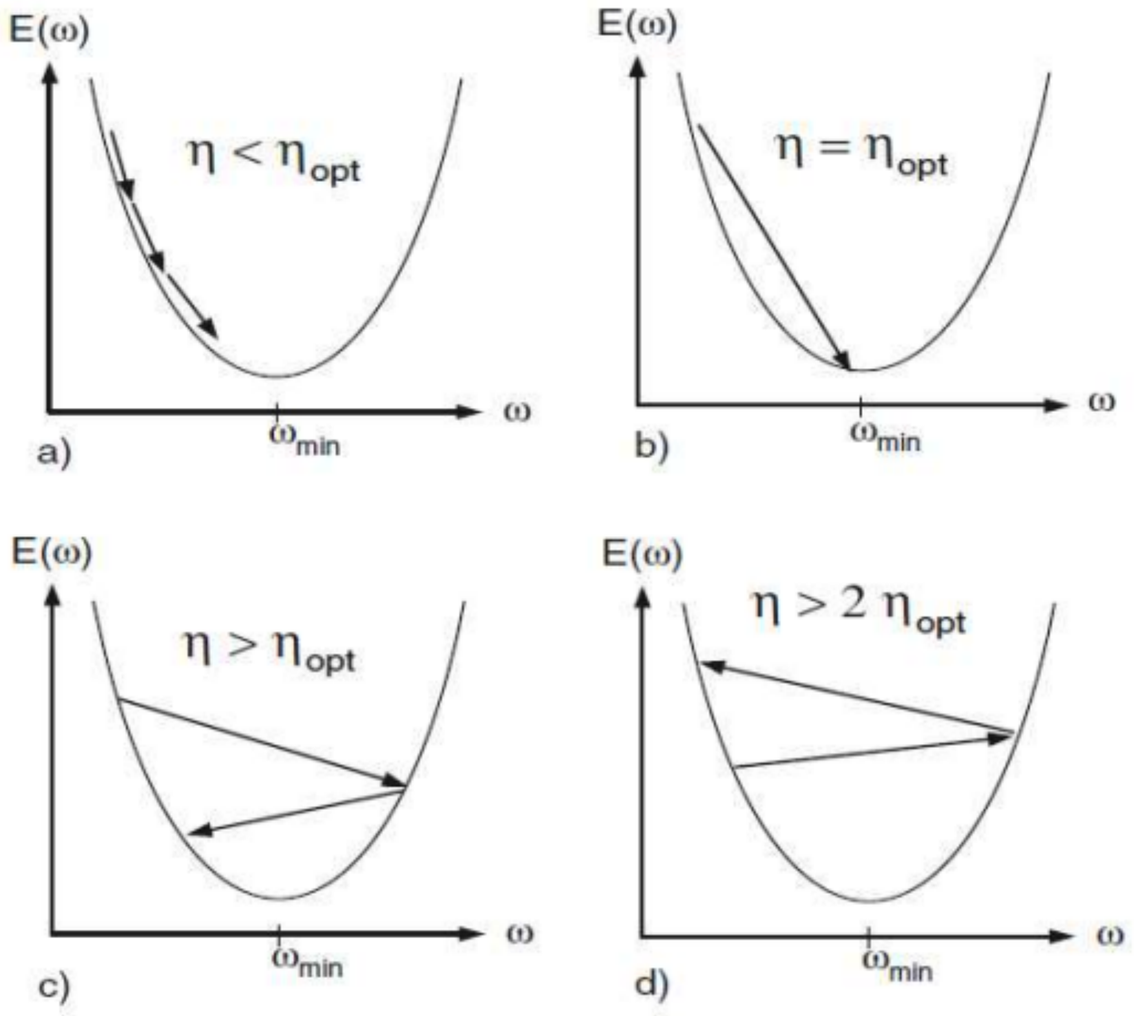
Learning rate

$$w_{t+1} = w_t - \eta \nabla \mathscr{L}(w_t)$$

η_{opt} ?

Hint: Taylor expansion around the current point

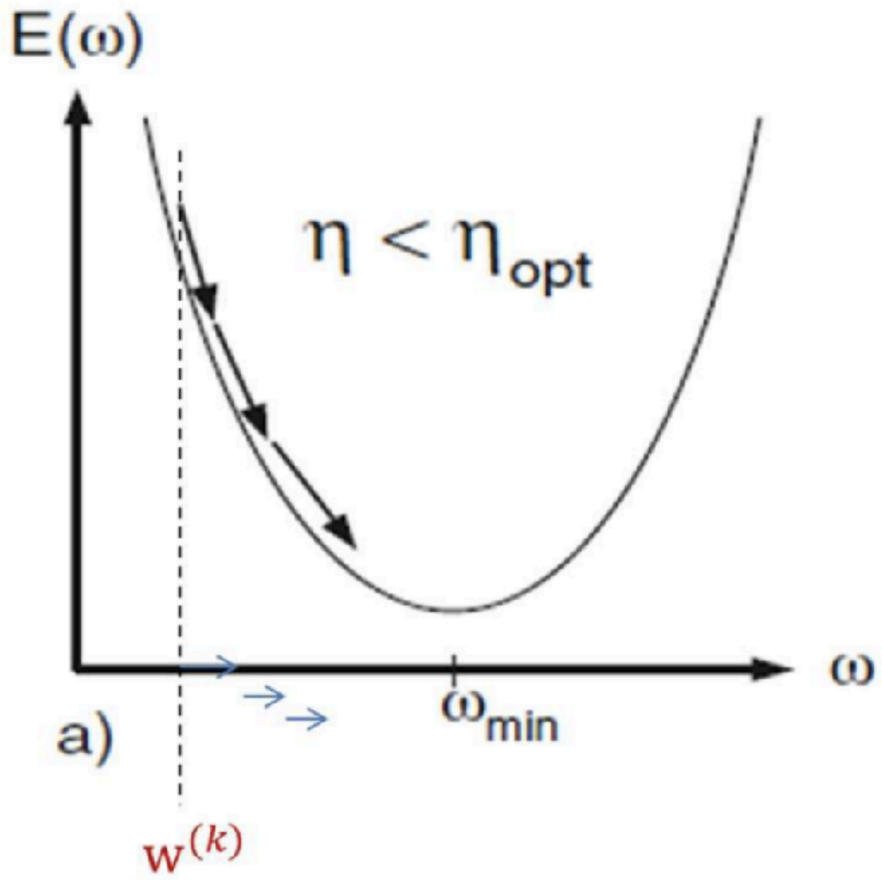
$$f(w) = \frac{1}{2} a w^2 + b w + c$$



From https://deeplearning.cs.cmu.edu/F24/index.html

Any quadratic objective may be written as:

$$E(w) = E(w_t) + E'(w_t)(w - w_t) + \frac{1}{2}E''(w_t)(w - w_t)^2$$

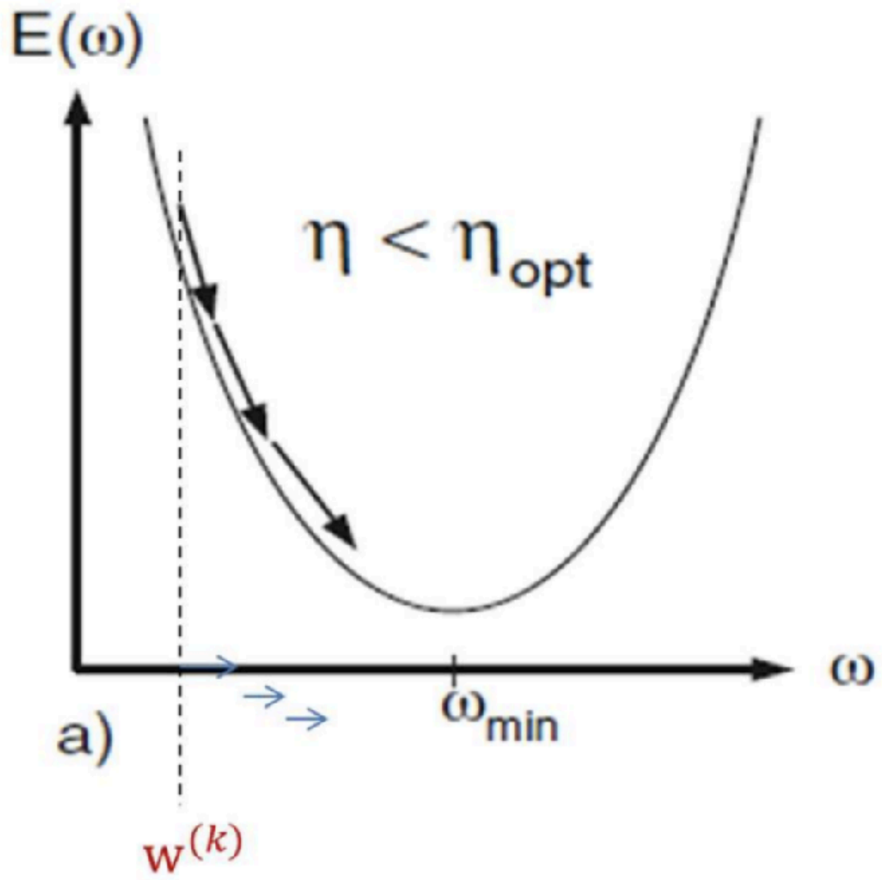


Any quadratic objective may be written as:

$$E(w) = E(w_t) + E'(w_t)(w - w_t) + \frac{1}{2}E''(w_t)(w - w_t)^2$$

Minimizing w.r. to *w* gives Newton's method:

$$w_{\min} = w_t - \left(E''(w_t)\right)^{-1} E'(w_t)$$



Any quadratic objective may be written as:

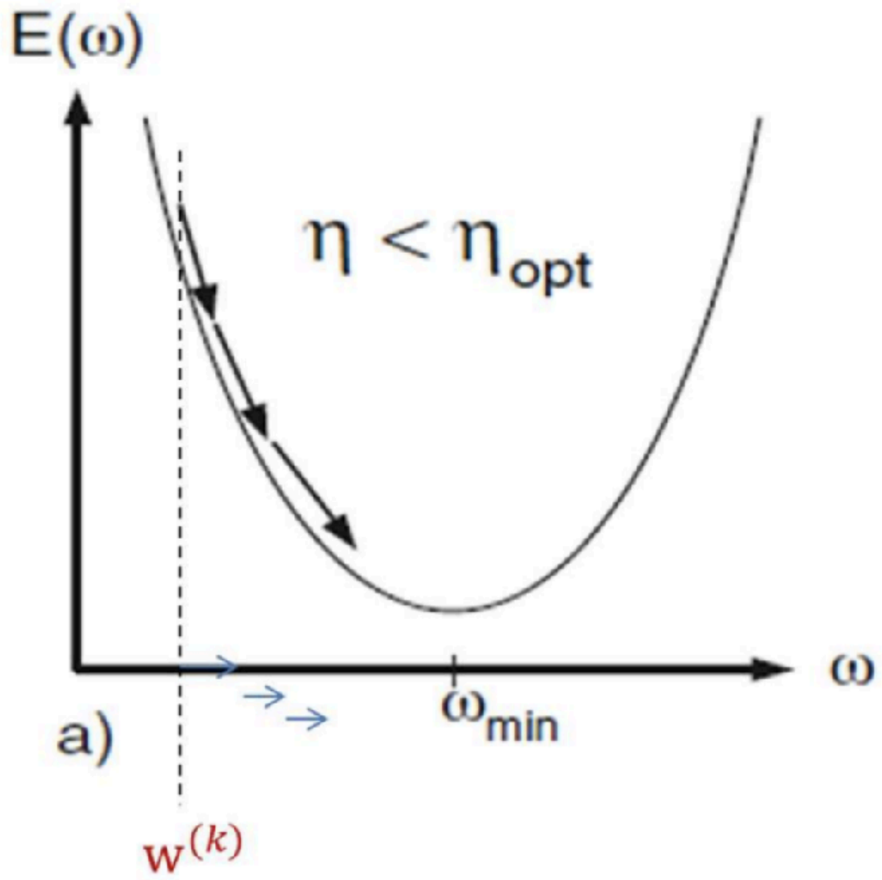
$$E(w) = E(w_t) + E'(w_t)(w - w_t) + \frac{1}{2}E''(w_t)(w - w_t)^2$$

Minimizing w.r. to *w* gives Newton's method:

$$w_{\min} = w_t - \left(E''(w_t)\right)^{-1} E'(w_t)$$

Compare with gradient descent:

$$w_{t+1} = w_t - \eta \frac{dE(w_t}{dw}$$



Any quadratic objective may be written as:

$$E(w) = E(w_t) + E'(w_t)(w - w_t) + \frac{1}{2}E''(w_t)(w - w_t)^2$$

Minimizing w.r. to *w* gives Newton's method:

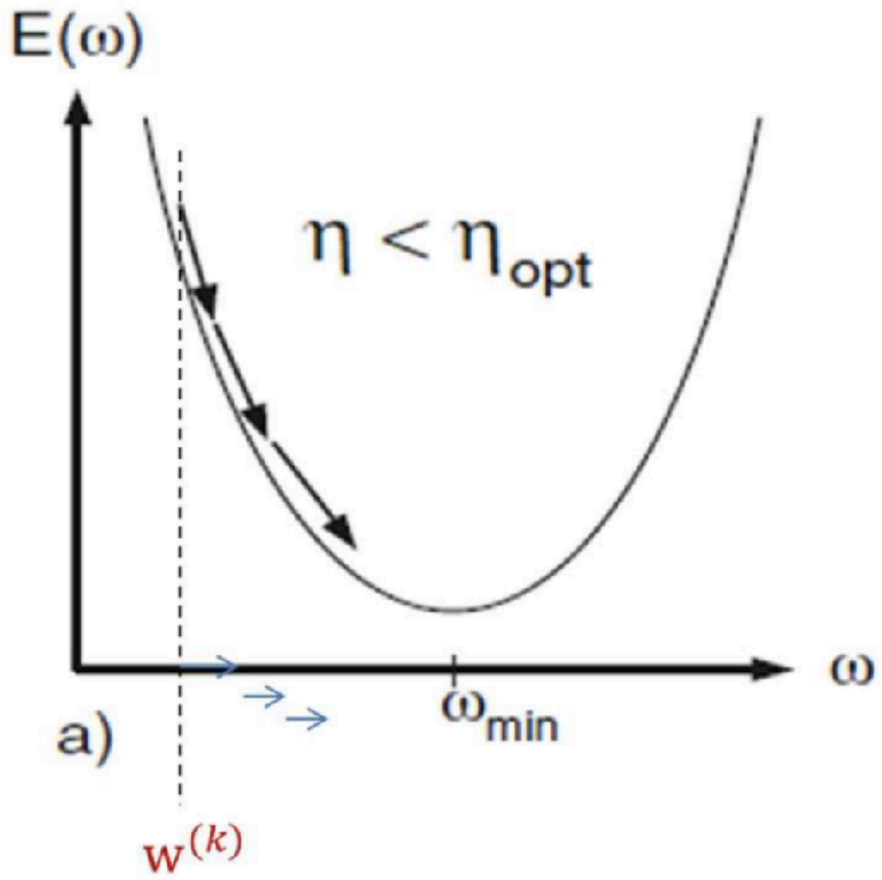
$$w_{\min} = w_t - \left(E''(w_t)\right)^{-1} E'(w_t)$$

Compare with gradient descent:

$$w_{t+1} = w_t - \eta \frac{dE(w_t}{dw}$$

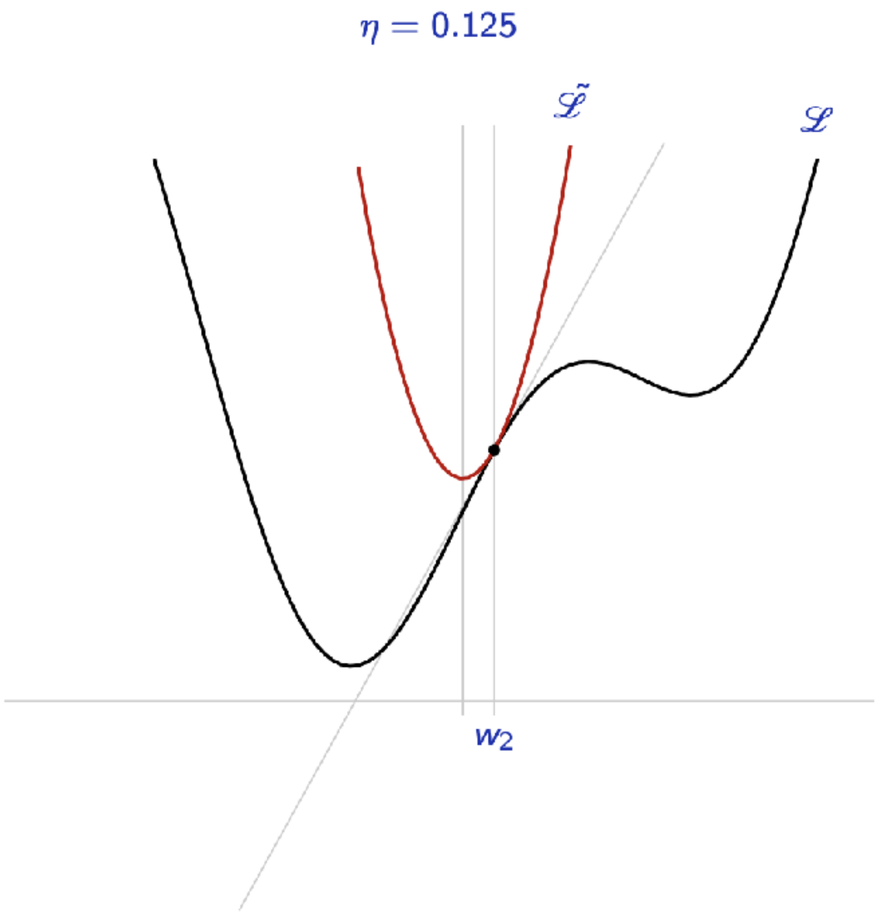
Can reach optimum in one step with:

$$\eta_{\text{opt}} = \left(E''(w_t)\right)^{-1}$$



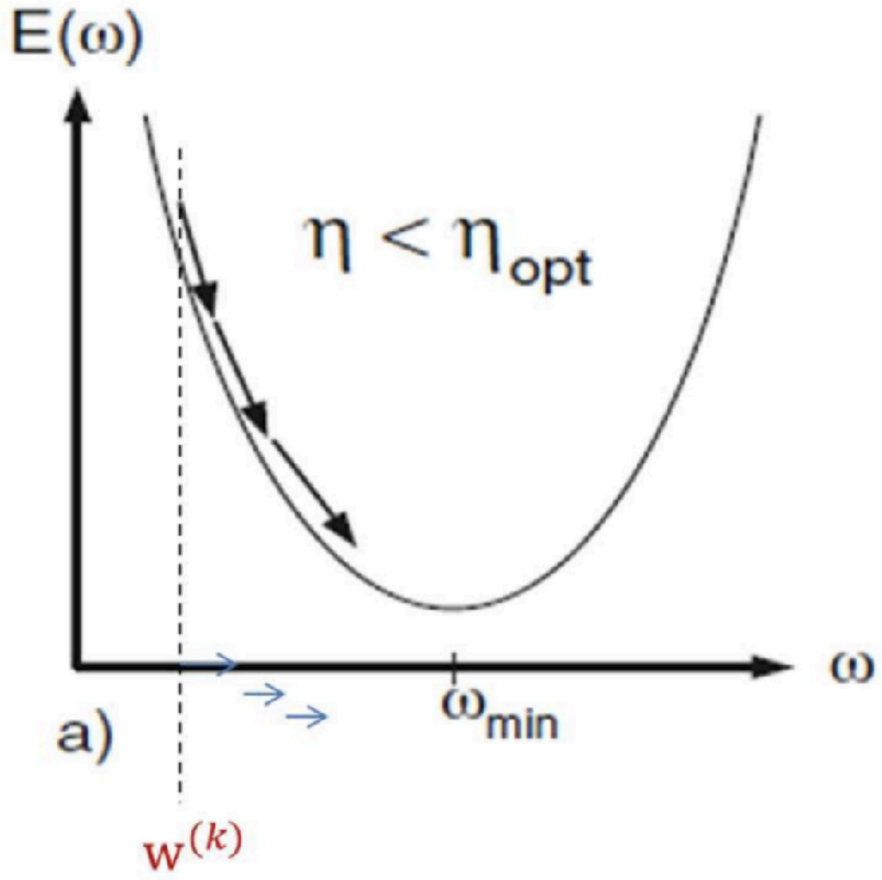
For a more complex function, we can locally approximate with a quadratic one, and move in small steps.

What can go wrong?



Learning rate heuristics

- Choose an initial learning rate
- When loss starts jumping around or not decrease, decay the learning rate



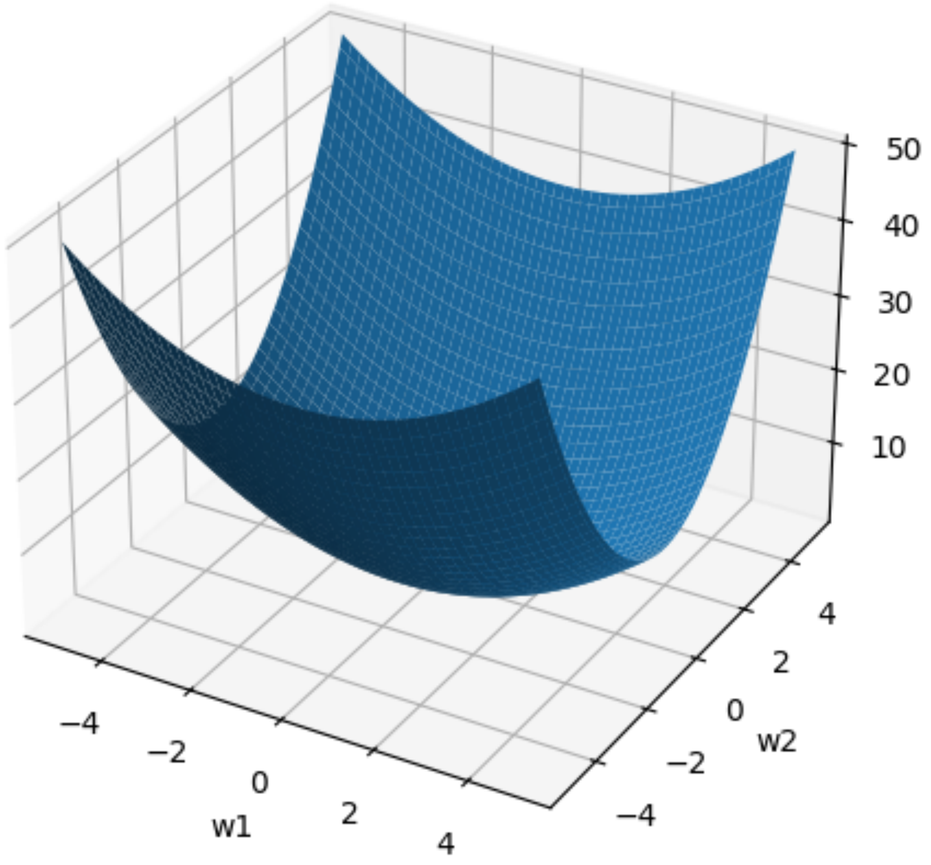
Linear decay

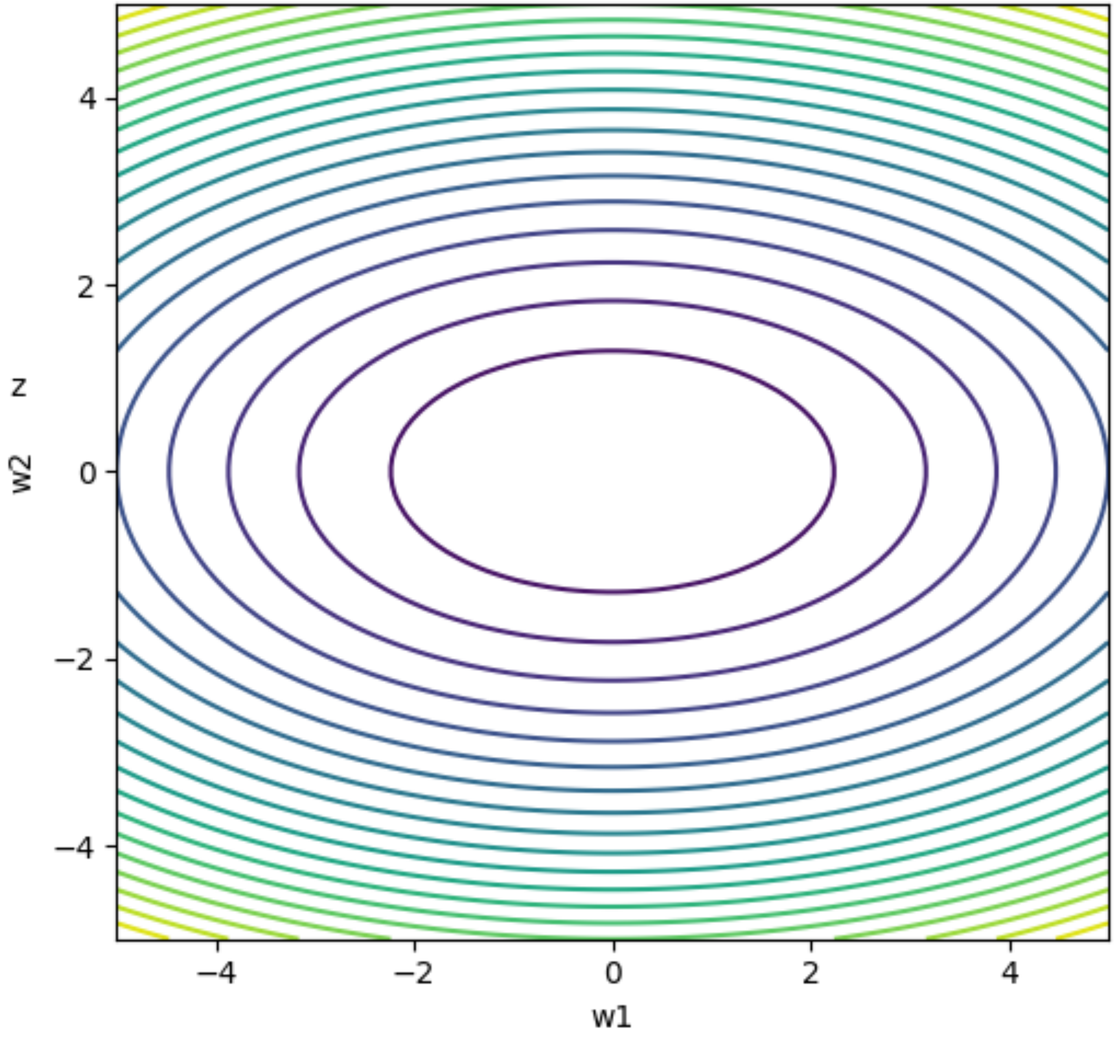
$$\eta_t = \frac{1}{t+1}\eta_0$$

Exponential decay

$$\eta_t = e^{-\beta t}\eta_0$$

$$\mathcal{L}(w) = \frac{1}{2}w_1^2 + \frac{3}{2}w_2^2$$



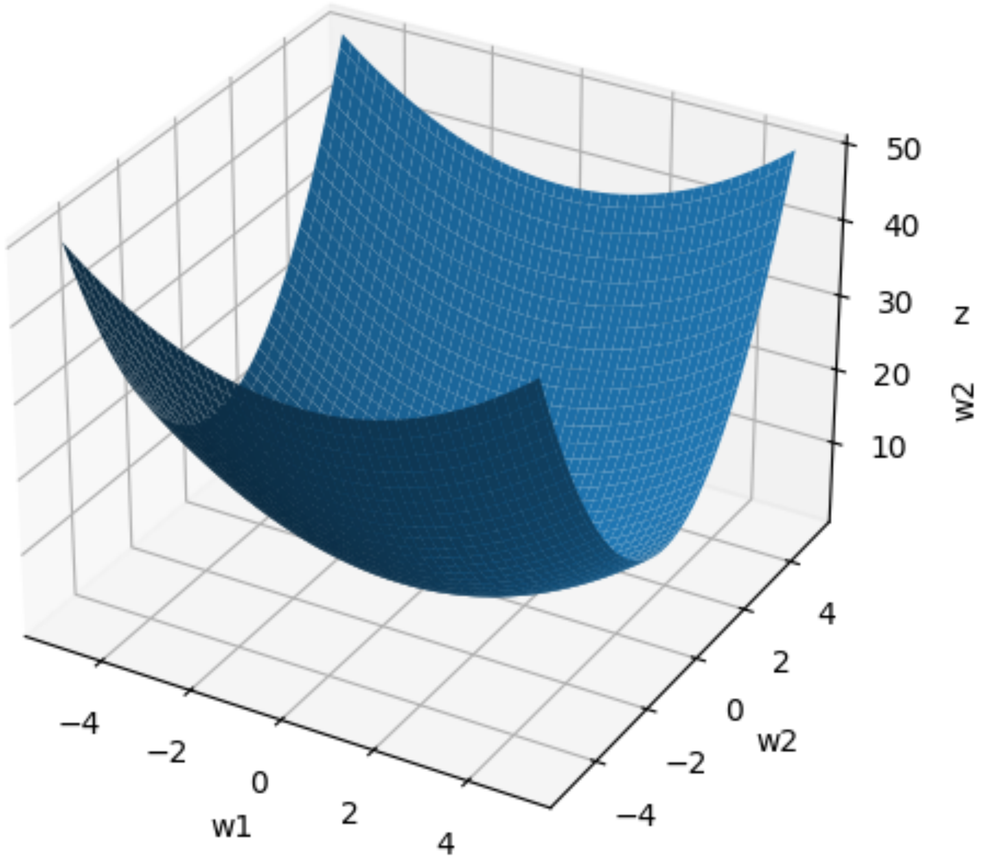


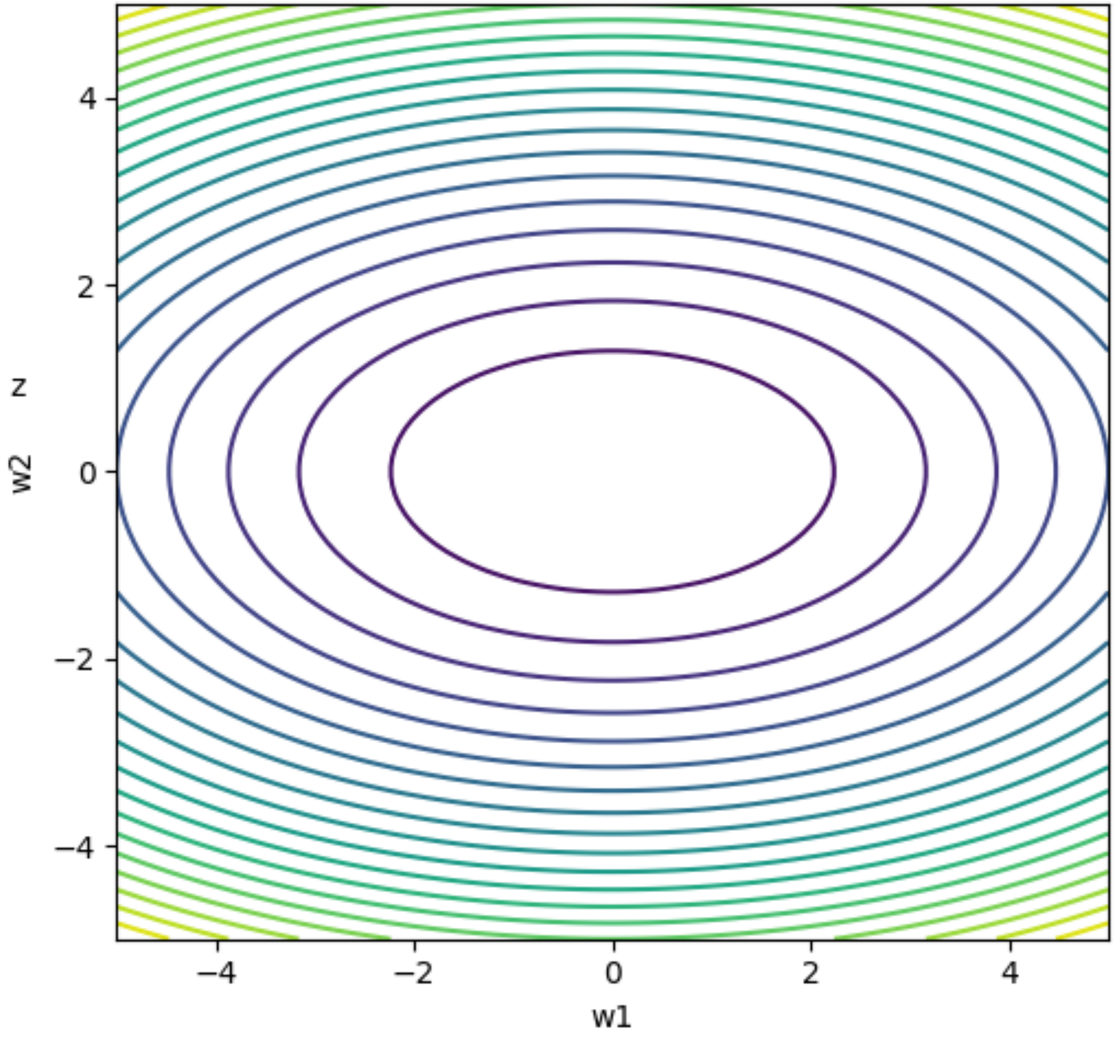
$$\mathcal{L}(w) = \frac{1}{2}w_1^2 + \frac{3}{2}w_2^2$$

$$w_{t+1,i} = w_{t,i} - \eta_i \nabla \mathcal{L}(w_{t,i})$$

$$\eta_{opt,1} = 1$$

$$\eta_{opt,2} = 0.33$$





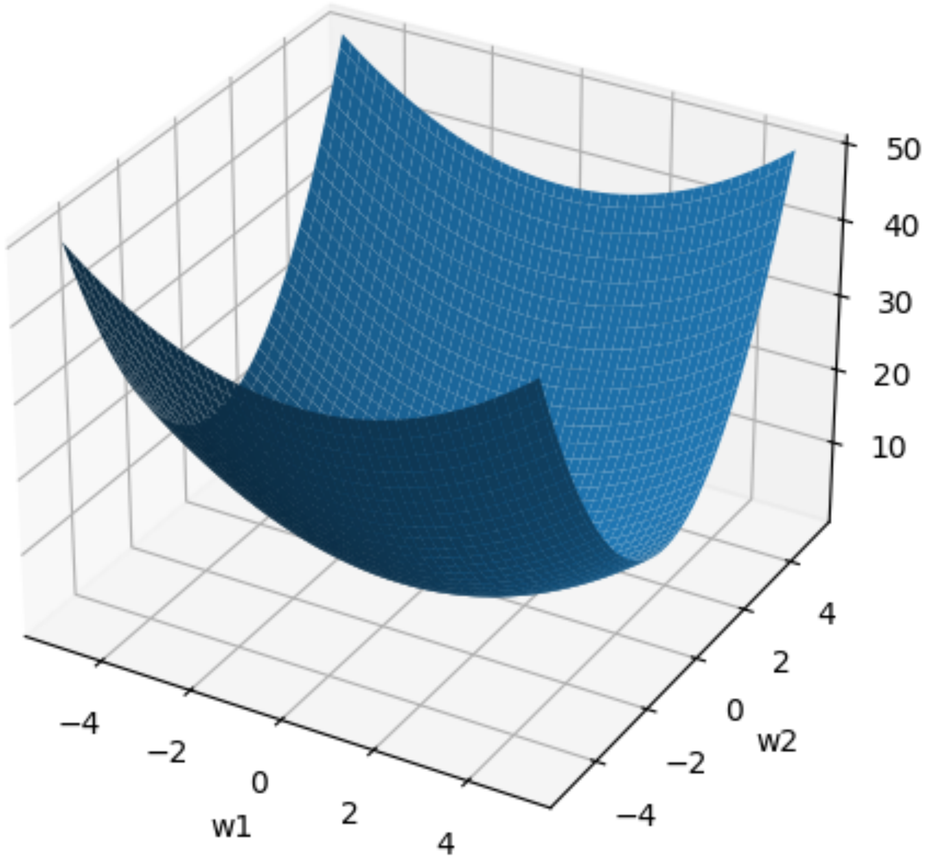
$$w_{t+1,i} = w_{t,i} - \eta_i \nabla \mathcal{L}(w_{t,i})$$

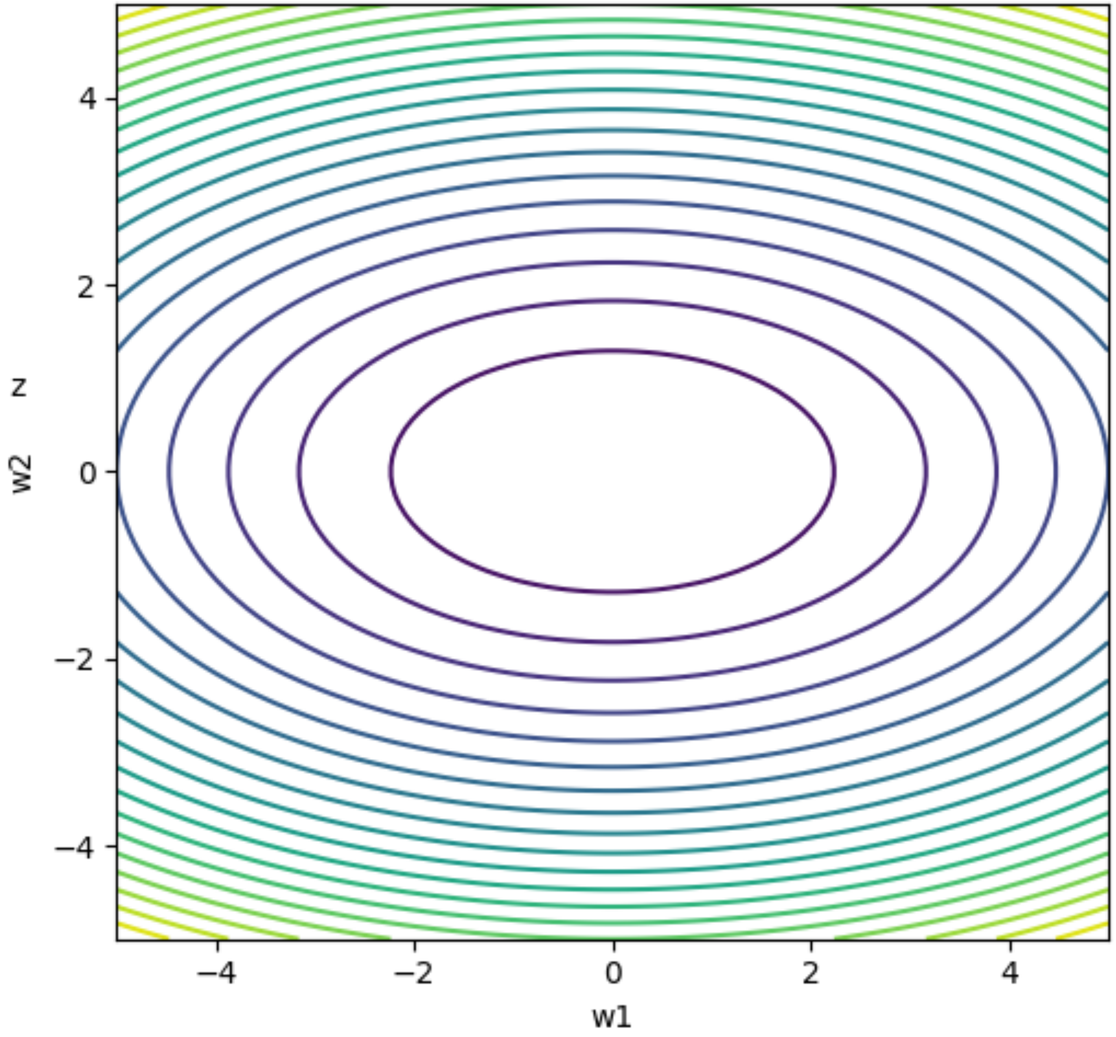
$$\eta_{1,opt} = 1$$

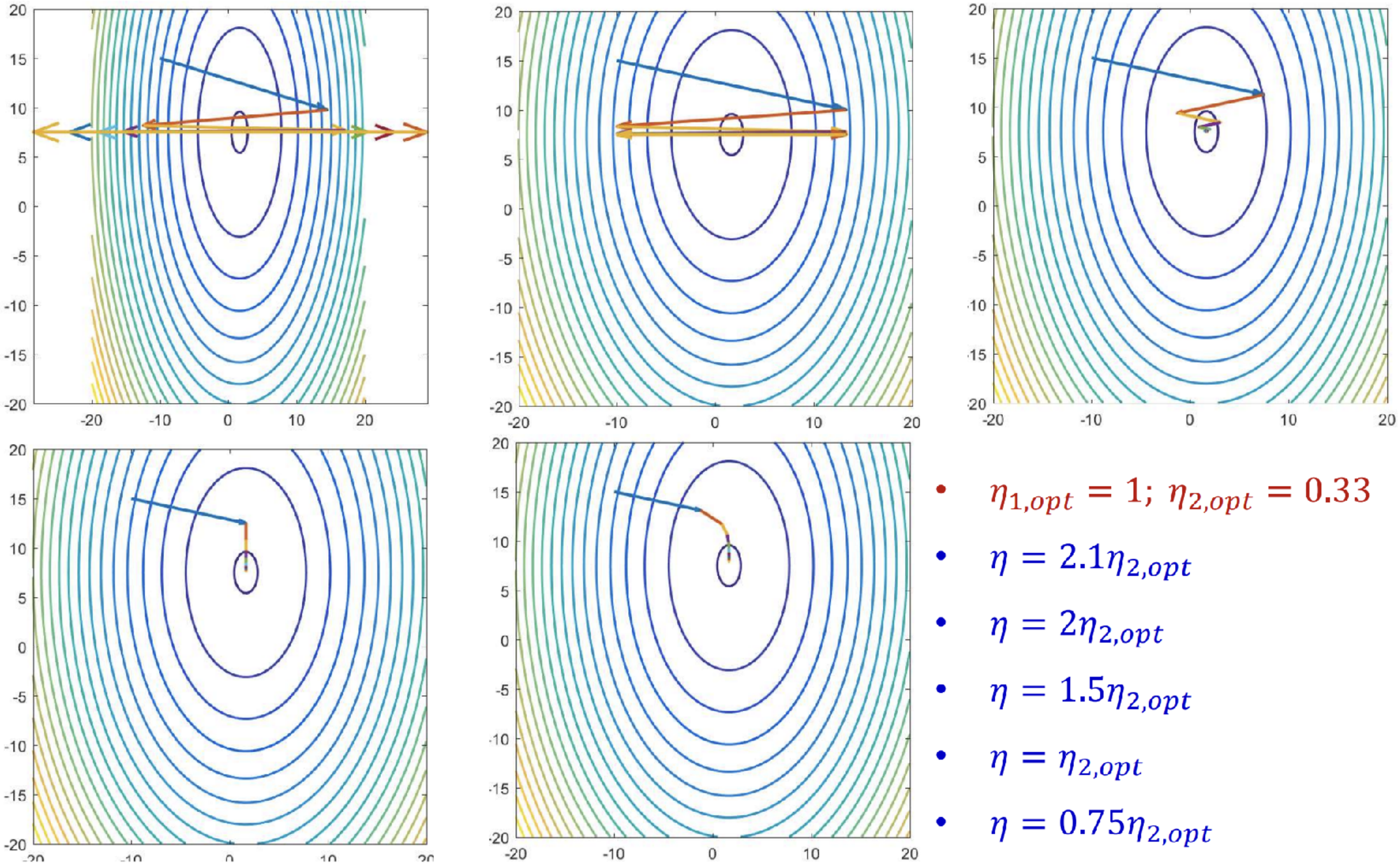
$$\eta_{2,opt} = 0.33$$

$$\neq$$

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

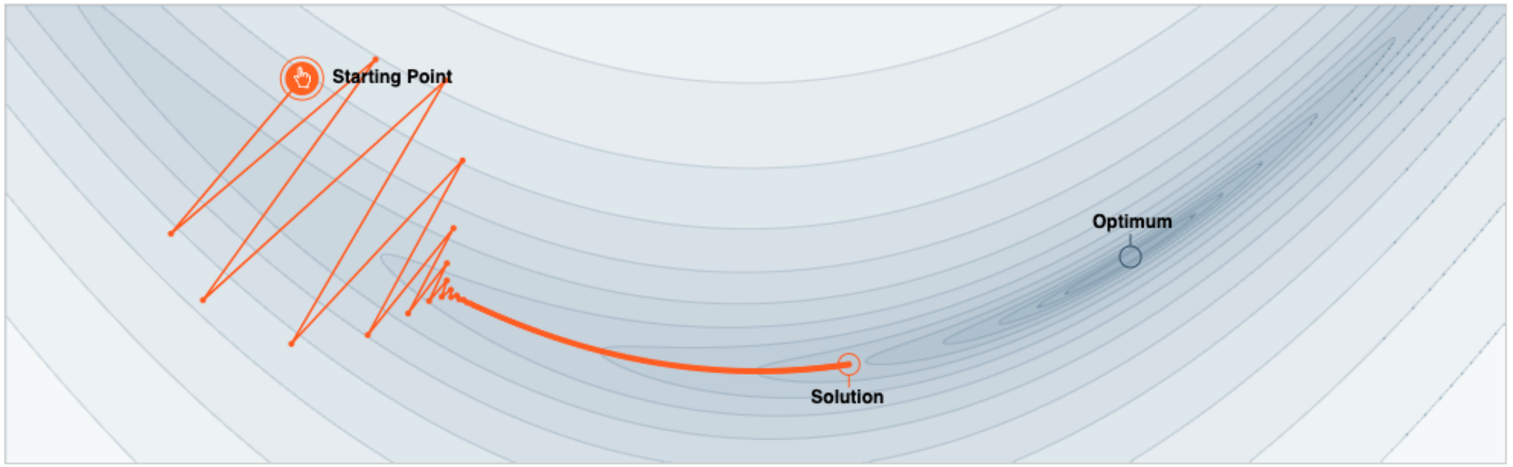






- $\eta_{1,opt} = 1;\ \eta_{2,opt} = 0.33$
- $\eta = 2.1\eta_{2,opt}$
- $\eta = 2\eta_{2,opt}$
- $\eta = 1.5\eta_{2,opt}$
- $\eta = \eta_{2,opt}$
- $\eta = 0.75\eta_{2,opt}$

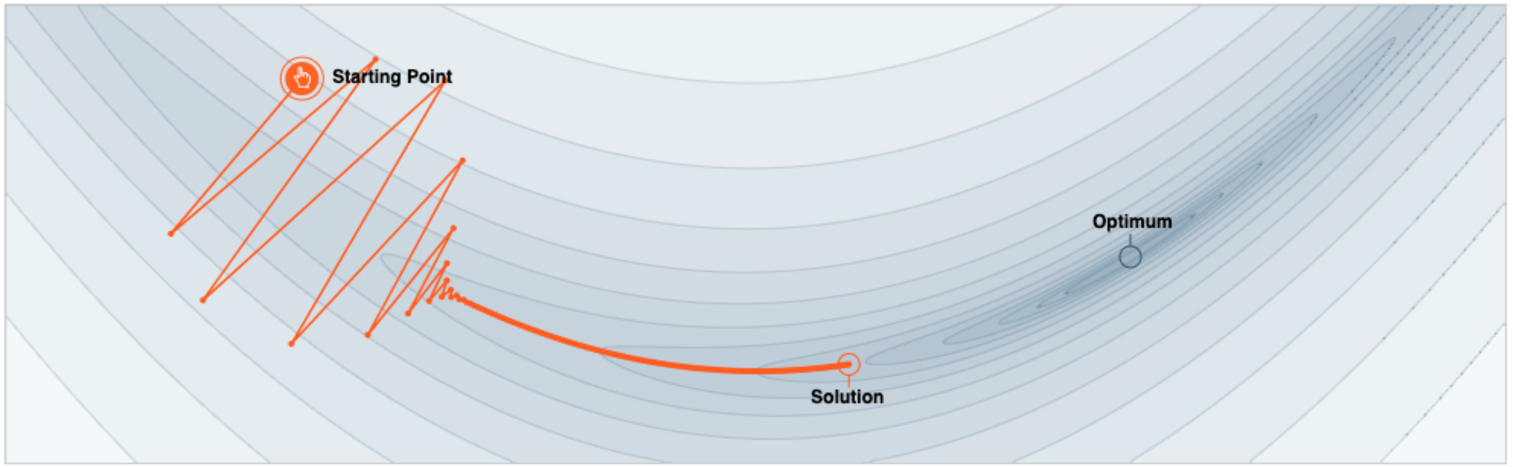
Momentum



Gradient descent

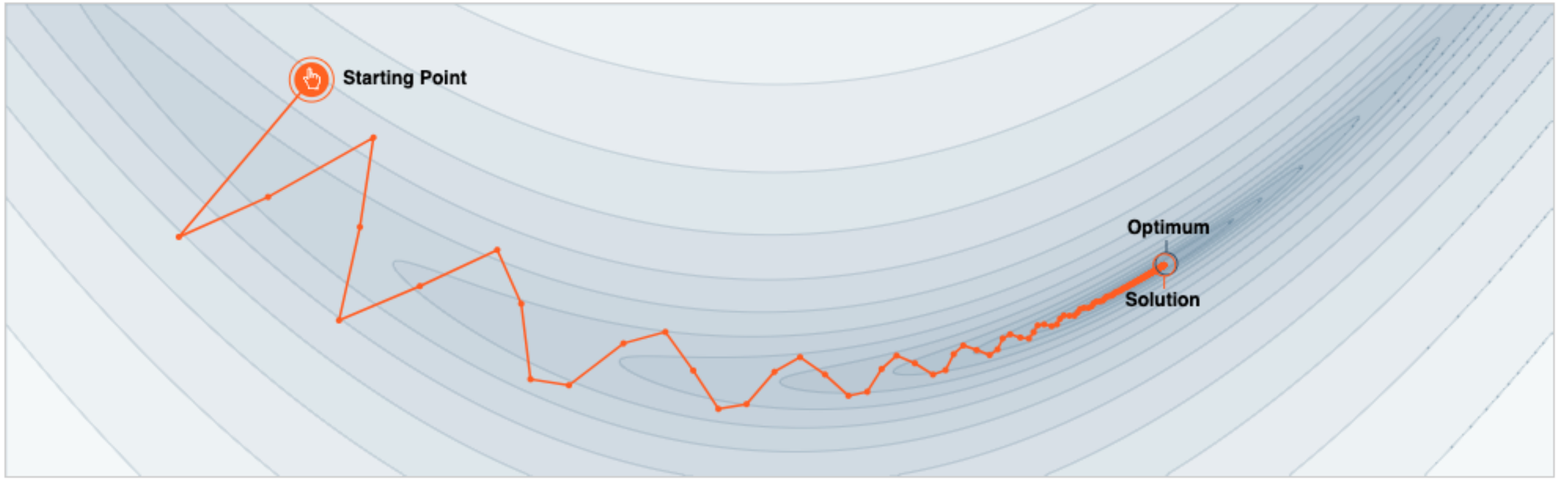
$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

https://distill.pub/2017/momentum/



Gradient descent

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$



Momentum

- Stabilizes
- Accelerates

$$v_{t+1} = \beta v_t + \eta \nabla \mathcal{L}(w_t)$$

$$w_{t+1} = w_t - v_{t+1}$$

https://distill.pub/2017/momentum/

Classic momentum

$$v_{t+1} = \beta v_t + \eta \nabla \mathcal{L}(w_t)$$

$$w_{t+1} = w_t - v_{t+1}$$

Classic momentum

$$v_{t+1} = \beta v_t + \eta \nabla \mathcal{L}(w_t)$$

$$w_{t+1} = w_t - v_{t+1}$$

Nesterov Accelerated Gradient

$$w'_t = w_t - \beta v_t$$

"Look ahead"

Classic momentum

$$v_{t+1} = \beta v_t + \eta \nabla \mathcal{L}(w_t)$$

$$w_{t+1} = w_t - v_{t+1}$$

Nesterov Accelerated Gradient

$$w'_t = w_t - \beta v_t$$

"Look ahead"

$$v_{t+1} = \beta v_t + \eta \nabla \mathcal{L}(w'_t)$$

Classic momentum

$$v_{t+1} = \beta v_t + \eta \nabla \mathcal{L}(w_t)$$

$$w_{t+1} = w_t - v_{t+1}$$

Nesterov Accelerated Gradient

$$w'_t = w_t - \beta v_t$$

"Look ahead"

$$v_{t+1} = \beta v_t + \eta \nabla \mathcal{L}(w'_t)$$

$$w_{t+1} = w_t - v_{t+1}$$

Classic momentum

$$v_{t+1} = \beta v_t + \eta \nabla \mathcal{L}(w_t)$$

$$w_{t+1} = w_t - v_{t+1}$$

Nesterov Accelerated Gradient

$$w'_t = w_t - \beta v_t$$

"Look ahead"

$$v_{t+1} = \beta v_t + \eta \nabla \mathcal{L}(w'_t)$$

$$w_{t+1} = w_t - v_{t+1}$$

Advantages:

- Faster convergence on smooth functions
- Better stability (often)
- Theoretically optimal for convex fn

Momentum and convergence

Consider quadratic loss function w diagonal Hessian

$$\mathscr{L}(w) = \frac{1}{2}\sum_i \lambda_i w_i^2 = \frac{1}{2} w^\top \Lambda w \implies \nabla \mathscr{L}(w) = \Lambda w$$ Gradient

$$w^* = 0$$ Minimizer is 0 by design

Rule of the thumb for fastest convergence:

$$2 = \eta(\lambda_{\max} + \lambda_{\min}) \Rightarrow \ \eta^* = \frac{2}{\lambda_{\max} + \lambda_{\min}}$$

(Compromise between “bouncing” on direction max and converging too slow on direction min)

Consider quadratic loss function w diagonal Hessian

$$\mathscr{L}(w) = \frac{1}{2}\sum_i \lambda_i w_i^2 = \frac{1}{2} w^\top \Lambda w \Longrightarrow \nabla \mathscr{L}(w) = \Lambda w$$ Gradient

$$w^* = 0$$ Minimizer is 0 by design

$$w_{t+1} = w_t - \eta \nabla \mathscr{L}(w_t) = (I - \eta\Lambda) w_t$$ SGD update

Rule of the thumb for fastest convergence:

$$2 = \eta(\lambda_{\max} + \lambda_{\min}) \Rightarrow \eta^* = \frac{2}{\lambda_{\max} + \lambda_{\min}}$$

(Compromise between “bouncing” on direction max and converging too slow on direction min)

Consider quadratic loss function w diagonal Hessian

$$\mathscr{L}(w) = \frac{1}{2}\sum_i \lambda_i w_i^2 = \frac{1}{2} w^\top \Lambda w \Longrightarrow \nabla \mathscr{L}(w) = \Lambda w$$ Gradient

$w^* = 0$ Minimizer is 0 by design

$$w_{t+1} = w_t - \eta \nabla \mathscr{L}(w_t) = (I - \eta\Lambda) w_t$$ SGD update

$$\Rightarrow w_{i,t+1} = (1 - \eta\lambda_i)\, w_{i,t} = (1 - \eta\lambda_i)^{(t+1)}\, w_{i,0}$$ unrolling from k=0

Rule of the thumb for fastest convergence:

$$2 = \eta(\lambda_{\max} + \lambda_{\min}) \Rightarrow \eta^* = \frac{2}{\lambda_{\max} + \lambda_{\min}}$$

(Compromise between “bouncing” on direction max and converging too slow on direction min)

Consider quadratic loss function w diagonal Hessian

$$\mathscr{L}(w) = \frac{1}{2}\sum_i \lambda_i w_i^2 = \frac{1}{2} w^\top \Lambda w \Longrightarrow \nabla \mathscr{L}(w) = \Lambda w$$ Gradient

$w^* = 0$ Minimizer is 0 by design

$$w_{t+1} = w_t - \eta \nabla \mathscr{L}(w_t) = (I - \eta\Lambda) w_t$$ SGD update

$$\Rightarrow w_{i,t+1} = (1 - \eta\lambda_i)\, w_{i,t} = (1 - \eta\lambda_i)^{(t+1)}\, w_{i,0}$$ unrolling from k=0

$$\Rightarrow w_{i,t} - w_i^* = (1 - \eta\lambda_i)^{t+1}(w_i^0 - w_i^*)$$ Since the minimizer is zero

Rule of the thumb for fastest convergence:

$$2 = \eta(\lambda_{\max} + \lambda_{\min}) \Rightarrow \eta^* = \frac{2}{\lambda_{\max} + \lambda_{\min}}$$

(Compromise between “bouncing” on direction max and converging too slow on direction min)

Consider quadratic loss function w diagonal Hessian

$$\mathscr{L}(w) = \frac{1}{2}\sum_i \lambda_i w_i^2 = \frac{1}{2} w^\top \Lambda w \Longrightarrow \nabla \mathscr{L}(w) = \Lambda w$$ Gradient

$w^* = 0$ Minimizer is 0 by design

$$w_{t+1} = w_t - \eta \nabla \mathscr{L}(w_t) = (I - \eta\Lambda) w_t$$ SGD update

$$\Rightarrow w_{i,t+1} = (1 - \eta\lambda_i)\, w_{i,t} = (1 - \eta\lambda_i)^{(t+1)}\, w_{i,0}$$ unrolling from k=0

$$\Rightarrow w_{i,t} - w_i^* = (1 - \eta\lambda_i)^{t+1}(w_i^0 - w_i^*)$$ Since the minimizer is zero

Rule of the thumb for fastest convergence:

$$2 = \eta(\lambda_{\max} + \lambda_{\min}) \Rightarrow \eta^* = \frac{2}{\lambda_{\max} + \lambda_{\min}}$$

(Compromise between “bouncing” on direction max and converging too slow on direction min)

Consider quadratic loss function w diagonal Hessian

$$\mathscr{L}(w) = \frac{1}{2}\sum_i \lambda_i w_i^2 = \frac{1}{2} w^\top \Lambda w \implies \nabla \mathscr{L}(w) = \Lambda w$$ Gradient

$$w^* = 0$$ Minimizer is 0 by design

$$w_{t+1} = w_t - \eta \nabla \mathscr{L}(w_t) = (I - \eta\Lambda) w_t$$ SGD update

$$\Rightarrow w_{i,t+1} = (1 - \eta\lambda_i)\, w_{i,t} = (1 - \eta\lambda_i)^{(t+1)}\, w_{i,0}$$ unrolling from k=0

$$\Rightarrow w_{i,t} - w_i^* = (1 - \eta\lambda_i)^{t+1}(w_i^0 - w_i^*)$$ Since the minimizer is zero

$$|1 - \eta\lambda_i| < 1 \Rightarrow 0 < \eta < \frac{2}{\lambda_i}$$ Condition for convergence

Rule of the thumb for fastest convergence:

$$2 = \eta(\lambda_{\max} + \lambda_{\min}) \Rightarrow \eta^* = \frac{2}{\lambda_{\max} + \lambda_{\min}}$$

(Compromise between “bouncing” on direction max and converging too slow on direction min)

Consider quadratic loss function w diagonal Hessian

$$\mathscr{L}(w) = \frac{1}{2}\sum_i \lambda_i w_i^2 = \frac{1}{2} w^\top \Lambda w \Longrightarrow \nabla \mathscr{L}(w) = \Lambda w$$ Gradient

$$w^* = 0$$ Minimizer is 0 by design

$$w_{t+1} = w_t - \eta \nabla \mathscr{L}(w_t) = (I - \eta\Lambda) w_t$$ SGD update

$$\Rightarrow w_{i,t+1} = (1 - \eta\lambda_i)\, w_{i,t} = (1 - \eta\lambda_i)^{(t+1)}\, w_{i,0}$$ unrolling from k=0

$$\Rightarrow w_{i,t} - w_i^* = (1 - \eta\lambda_i)^{t+1}(w_i^0 - w_i^*)$$ Since the minimizer is zero

$$|1 - \eta\lambda_i| < 1 \Rightarrow 0 < \eta < \frac{2}{\lambda_i}$$ Condition for convergence

$$0 < \eta < \frac{2}{\lambda_{\max}}$$ Global condition for convergence

Rule of the thumb for fastest convergence:

$$2 = \eta(\lambda_{\max} + \lambda_{\min}) \Rightarrow \eta^* = \frac{2}{\lambda_{\max} + \lambda_{\min}}$$

(Compromise between “bouncing” on direction max and converging too slow on direction min)

Consider quadratic loss function w diagonal Hessian

$$\mathscr{L}(w) = \frac{1}{2}\sum_i \lambda_i w_i^2 = \frac{1}{2} w^\top \Lambda w \Longrightarrow \nabla \mathscr{L}(w) = \Lambda w$$ Gradient

$w^* = 0$ Minimizer is 0 by design

$$w_{t+1} = w_t - \eta \nabla \mathscr{L}(w_t) = (I - \eta\Lambda) w_t$$ SGD update

$$\Rightarrow w_{i,t+1} = (1 - \eta\lambda_i)\, w_{i,t} = (1 - \eta\lambda_i)^{(t+1)}\, w_{i,0}$$ unrolling from k=0

$$\Rightarrow w_{i,t} - w_i^* = (1 - \eta\lambda_i)^{t+1}(w_i^0 - w_i^*)$$ Since the minimizer is zero

$$|1 - \eta\lambda_i| < 1 \Rightarrow 0 < \eta < \frac{2}{\lambda_i}$$ Condition for convergence

$$0 < \eta < \frac{2}{\lambda_{\max}}$$ Global condition for convergence

Rule of the thumb for fastest convergence:

$$2 = \eta(\lambda_{\max} + \lambda_{\min}) \Rightarrow \eta^* = \frac{2}{\lambda_{\max} + \lambda_{\min}}$$

(Compromise between "bouncing" on direction max and converging too slow on direction min)

$$\rho^* = |1 - \eta^*\lambda_{\min}| = 1 - \frac{2\lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\kappa - 1}{\kappa + 1}$$ Contraction factor

$$\mathcal{L}(w) = \frac{1}{2}\sum_i \lambda_i w_i^2$$

SGD

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

Momentum

$$\left\{ \begin{array}{l} v_{t+1} = \beta v_t + \nabla \mathcal{L}(w_t) \\ w_{t+1} = w_t - \eta v_{t+1} \end{array} \right\}$$

$$w_i^k - w_i^* = (1 - \eta\lambda_i)^{k+1}(w_i^0 - w_i^*)$$

$$\mathcal{L}(w^k) - \mathcal{L}(w^*) = \sum (1 - \eta\lambda_i)^{2k}\lambda_i(w_i^0 - w_i^*)^2$$

Converges if $0 < \eta < \frac{2}{\lambda_i}$

With rate $\frac{k-1}{k+1}$ $\quad k = \frac{\lambda_1}{\lambda_n}$

$$\mathcal{L}(w) = \frac{1}{2}\sum_i \lambda_i w_i^2$$

SGD

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

$$w_i^k - w_i^* = (1 - \eta\lambda_i)^{k+1}(w_i^0 - w_i^*)$$

$$\mathcal{L}(w^k) - \mathcal{L}(w^*) = \sum (1 - \eta\lambda_i)^{2k}\lambda_i(w_i^0 - w_i^*)^2$$

Momentum

$$\left\{ \begin{array}{l} v_{t+1} = \beta v_t + \nabla \mathcal{L}(w_t) \\ w_{t+1} = w_t - \eta v_{t+1} \end{array} \right\}$$

$$\cdots$$

Converges if $0 < \eta < \frac{2}{\lambda_i}$ $\qquad$ $0 < \eta < \frac{2 + 2\beta}{\lambda_i}$

With rate $\frac{k-1}{k+1}$ $\qquad$ $k = \frac{\lambda_1}{\lambda_n}$ $\qquad$ $\frac{\sqrt{k}-1}{\sqrt{k}+1}$

$$\mathcal{L}(w) = \frac{1}{2}\sum_i \lambda_i w_i^2$$

	SGD	Momentum
Converges if	$0 < \eta < \frac{2}{\lambda_i}$	$0 < \eta < \frac{2+2\beta}{\lambda_i}$
With rate	$\frac{k-1}{k+1}$	$\frac{\sqrt{k}-1}{\sqrt{k}+1}$

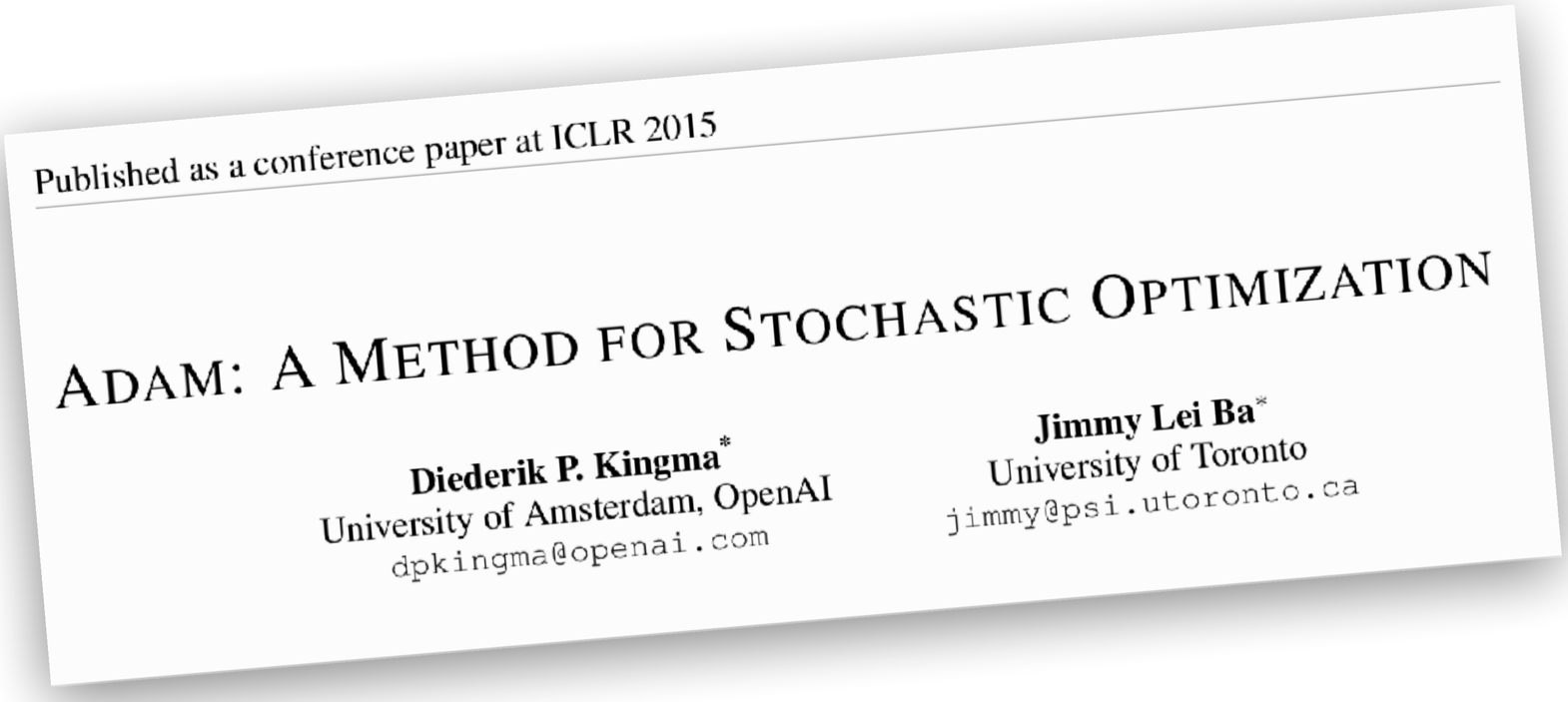
$$k = \frac{\lambda_1}{\lambda_n}$$

The convergence rate depends on the local curvature of error function.

Different parameters have different curvature.

So, we should modify learning rate separately for each parameter (works in practice, even if Hessian is not diagonal)

Adam



Published as a conference paper at ICLR 2015

ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

Diederik P. Kingma*
University of Amsterdam, OpenAI
dpkingma@openai.com

Jimmy Lei Ba*
University of Toronto
jimmy@psi.utoronto.ca

Idea 1: direction-specific scaling

$$g_t = \frac{\partial \mathscr{L}(w^t)}{\partial w}$$

Gradient

$$s_i^{t+1} = \beta_1 s_i^t + (1 - \beta_1) g_{t,i}$$

Momentum
(EMA of grad)

$$w_i^{t+1} = w_i^t - \eta_i s_i^{t+1}$$

Weights update

Idea 1: direction-specific scaling

$$g_t = \frac{\partial \mathscr{L}(w^t)}{\partial w}$$

Gradient

$$s_i^{t+1} = \beta_1 s_i^t + (1 - \beta_1) g_{t,i}$$

Momentum
(EMA of grad)

$$w_i^{t+1} = w_i^t - \eta_i s_i^{t+1}$$

Weights update

Direction-specific scaling … need method to estimate

Idea 2: estimate gradient variability

If the gradient is highly variable, then we need to take small steps…

$$g_t = \frac{\partial \mathscr{L}(w^t)}{\partial w}$$

Gradient

$$s_i^{t+1} = \beta_1 s_i^t + (1 - \beta_1) g_{t,i}$$

Momentum
(Moving ave of grad)

$$r_i^{t+1} = \beta_2 r_i^t + (1 - \beta_2) g_{t,i}^2$$

Moving ave of square grad
= estimate of grad variance

Idea 3: correct for bias of EMA

$$s^t = \beta_1 s_i^{t-1} + (1-\beta_1) g_{t,i} = (1-\beta_1) \sum_{k=1}^{t} \beta_1^{t-k} g_k$$

Unroll s^t

(Notation: $\mu = \mathbb{E}[g]$ mean of gradient)

$$\mathbb{E}[s^t] = (1-\beta_1) \sum_{k=1}^{t} \beta_1^{t-k} \mu = (1-\beta_1^t)\mu$$

Biased estimate

$$\bar{s}^t = \frac{s^t}{1-\beta_1^t}, \qquad \bar{r}^t = \frac{r^t}{1-\beta_2^t}$$

Unbiased estimates

Adam's recipe:

Gradient

$$g_t = \frac{\partial \mathscr{L}(w^t)}{\partial w}$$

Moving ave of grad

$$s_i^{t+1} = \beta_1 s_i^t + (1 - \beta_1) g_{t,i}$$

$$\bar{s}^t = \frac{s^t}{1 - \beta_1^t}$$

Estimate of grad variance

$$r_i^{t+1} = \beta_2 r_i^t + (1 - \beta_2) g_{t,i}^2$$

$$\bar{r}^t = \frac{r^t}{1 - \beta_2^t}$$

Weight update

$$w_i^{t+1} = w_i^t - \eta \frac{\bar{s}_i^{t+1}}{\sqrt{\bar{r}_i^{t+1}} + \delta}$$

To avoid explosions

Parameters Initialization

Where do we start with the weights?

Idea 1: let them all be the same value.

What can go wrong?

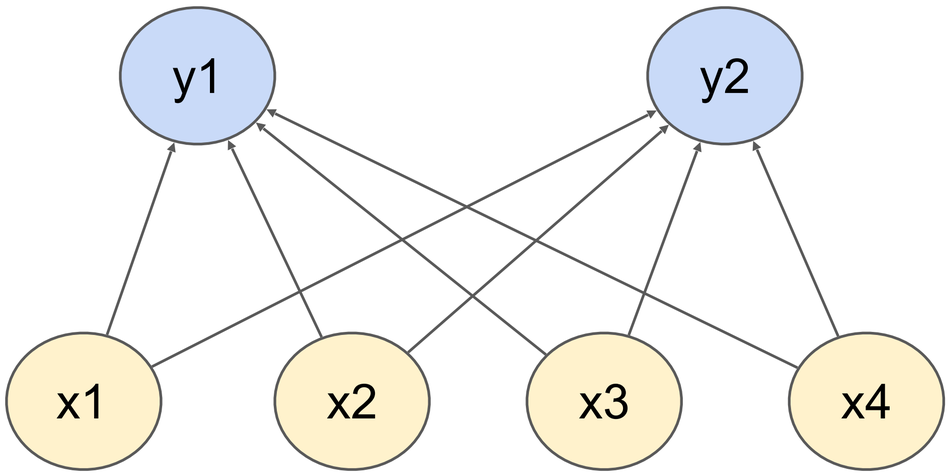
Parameters Initialization

Where do we start with the weights?

Idea 1: let them all be the same value.

What can go wrong?

y1 and y2 have the same inputs, with the same weights. The network will have no incentive to **break the symmetry**!

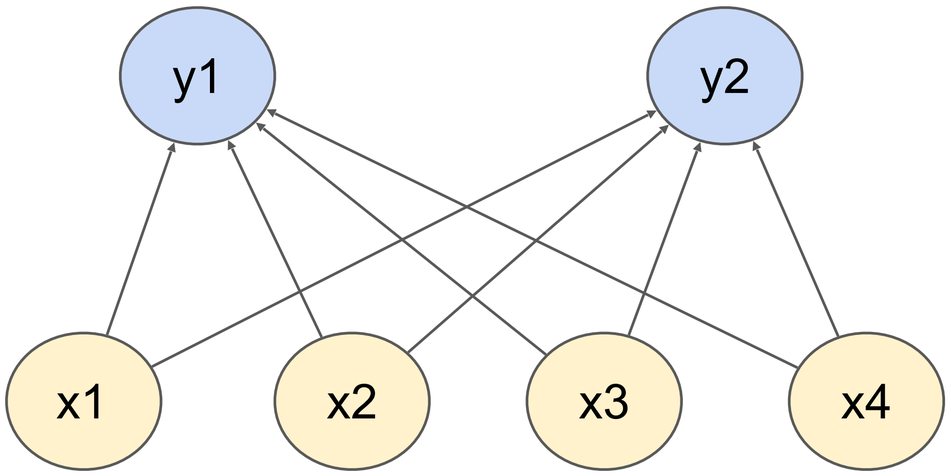


Parameters Initialization

Idea 2: Randomly distributed - Gaussian to make it easy

Let's stick with zero mean. How do we pick variance?

$$y_i = \sum_{j=1}^{n_{in}} w_{ij} x_j$$



Parameters Initialization

Idea 2: Randomly distributed - Gaussian to make it easy

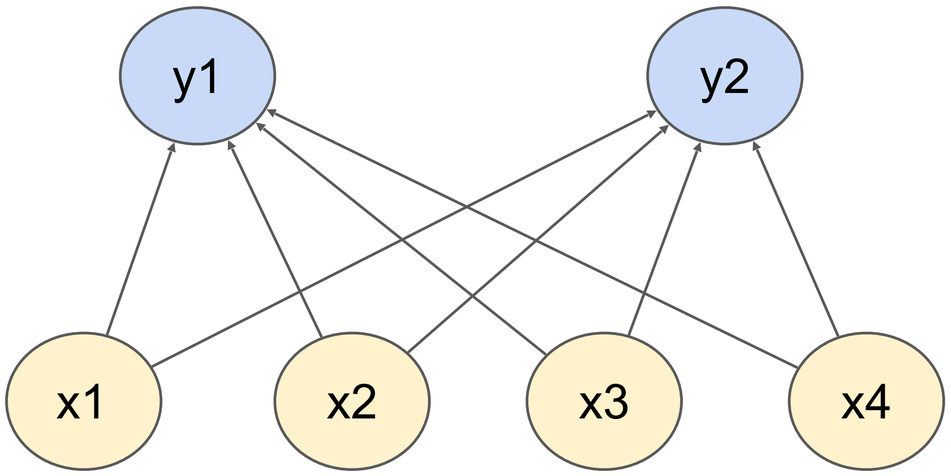
$$y_i = \sum_{j=1}^{n_{in}} w_{ij} x_j$$

Let's stick with zero mean. How do we pick variance?

If inputs have zero mean and variance σ_x^2

$$E(y_i) = 0$$

$$Var(y_i) = n_{in} \sigma_w^2 \sigma_x^2$$



Parameters Initialization

Idea 2: Randomly distributed - Gaussian to make it easy

$$y_i = \sum_{j=1}^{n_{in}} w_{ij} x_j$$

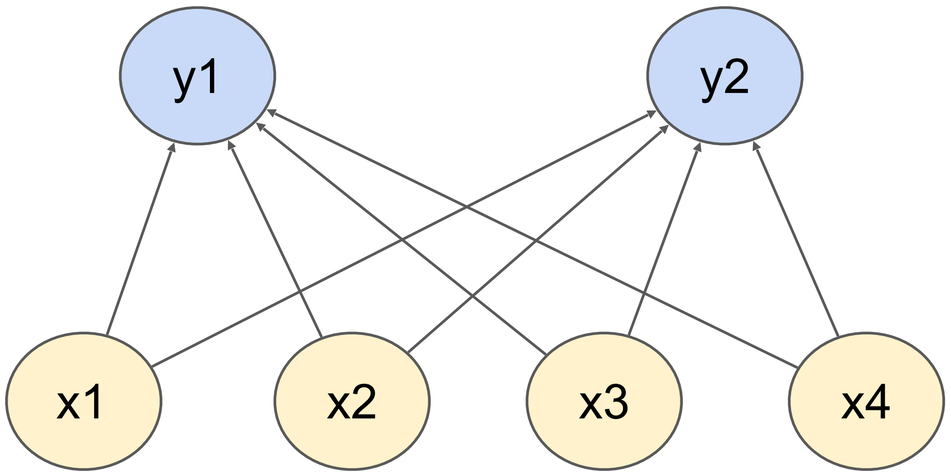
Let's stick with zero mean. How do we pick variance?

If inputs have zero mean and variance σ_x^2

$$E(y_i) = 0$$

$$Var(y_i) = n_{in} \sigma_w^2 \sigma_x^2$$

To keep layer variance constant: $\sigma_w^2 = \frac{1}{n_{in}}$



Parameters Initialization

Idea 2: Randomly distributed - Gaussian to make it easy

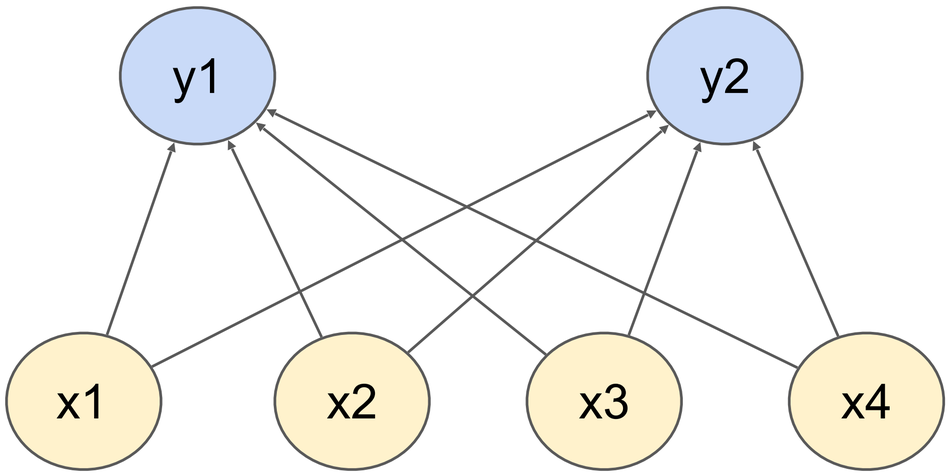
$$y_i = \sum_{j=1}^{n_{in}} w_{ij} x_j$$

Let's stick with zero mean. How do we pick variance?

If inputs have zero mean and variance σ_x^2

$$E(y_i) = 0$$

$$Var(y_i) = n_{in} \sigma_w^2 \sigma_x^2$$



To keep layer variance constant: $\sigma_w^2 = \frac{1}{n_{in}}$

To also keep gradient variance constant: $\sigma_w^2 = \frac{1}{n_{out}}$

Parameters Initialization

Idea 2: Randomly distributed - Gaussian to make it easy

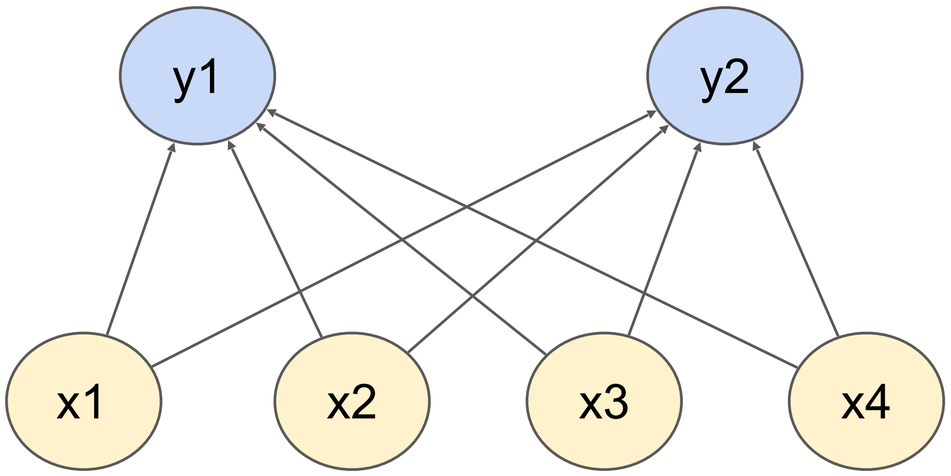
$$y_i = \sum_{j=1}^{n_{in}} w_{ij} x_j$$

Let's stick with zero mean. How do we pick variance?

If inputs have zero mean and variance σ_x^2

$$E(y_i) = 0$$

$$Var(y_i) = n_{in} \sigma_w^2 \sigma_x^2$$



To keep layer variance constant: $\sigma_w^2 = \frac{1}{n_{in}}$

To also keep gradient variance constant: $\sigma_w^2 = \frac{1}{n_{out}}$

Idea 3: Xavier's Initialization $\sigma_w^2 = \frac{2}{n_{in} + n_{out}}$

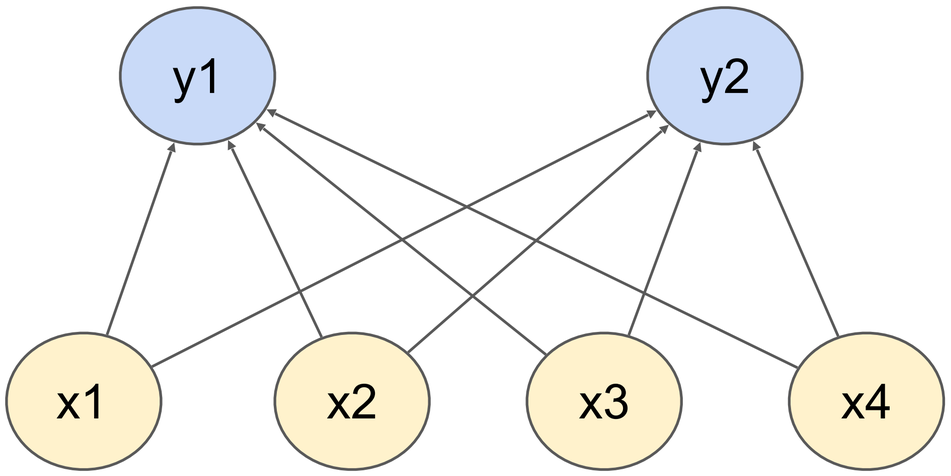
Parameters Initialization

We ignored the nonlinearities… for example, with ReLU

$$y_i = \sigma\left(\sum_{j=1}^{n_{in}} w_{ij} x_j\right)$$

$$E(y_i) = 0$$

$$Var(y_i) = \frac{n_{in}}{2} \sigma_w^2 \sigma_x^2$$



Idea 4 : Kaiming-He Initialization

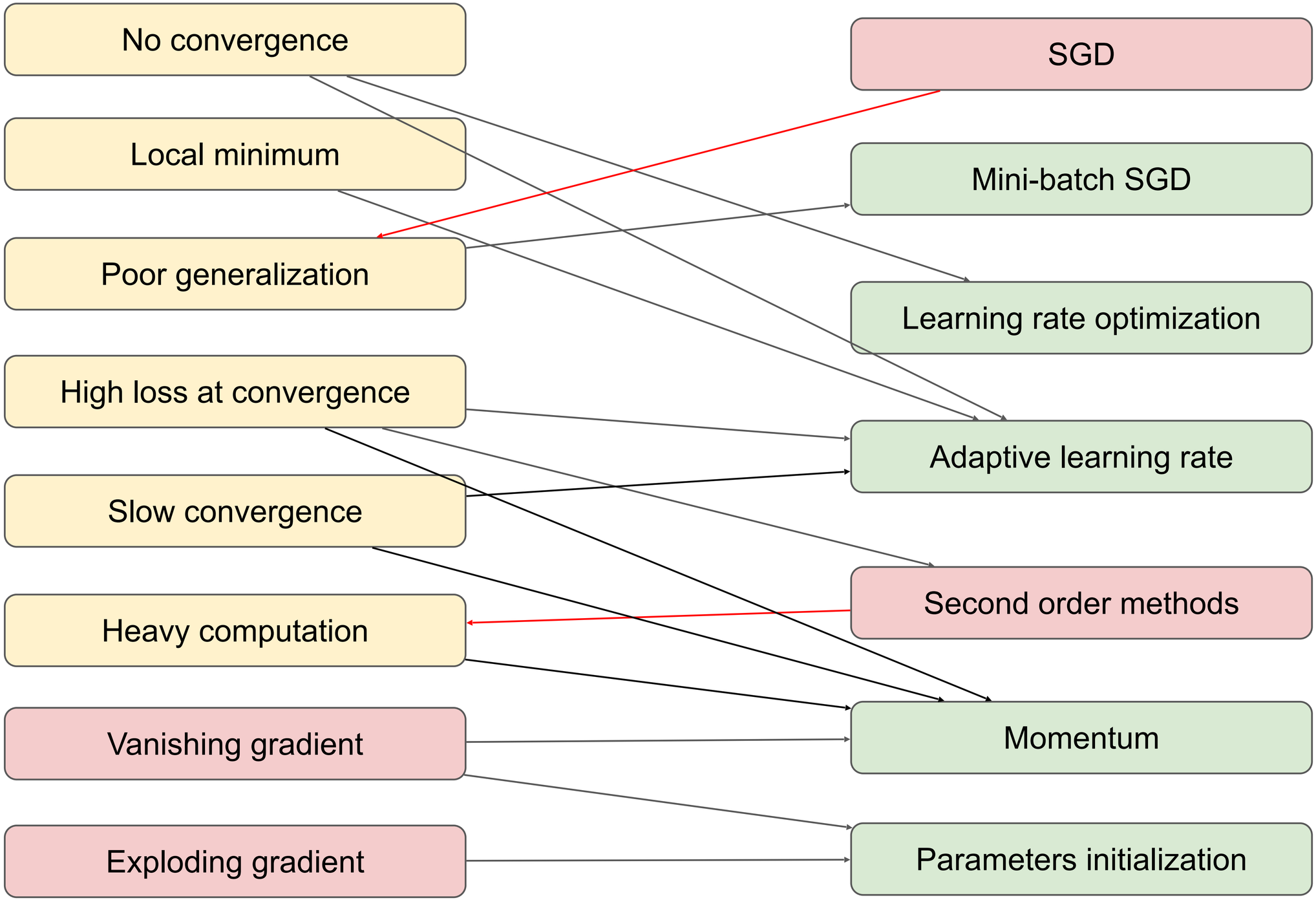
$$\sigma_w^2 = \frac{2}{n_{in}}$$

Recap

- No convergence
- Local minimum
- Poor generalization
- High loss at convergence
- Slow convergence
- Heavy computation
- Vanishing gradient
- Exploding gradient

- SGD
- Mini-batch SGD
- Learning rate optimization
- Adaptive learning rate
- Second order methods
- Momentum
- Parameters initialization

Recap


No convergence
Local minimum
Poor generalization
High loss at convergence
Slow convergence
Heavy computation
Vanishing gradient
Exploding gradient
SGD
Mini-batch SGD
Learning rate optimization
Adaptive learning rate
Second order methods
Momentum
Parameters initialization